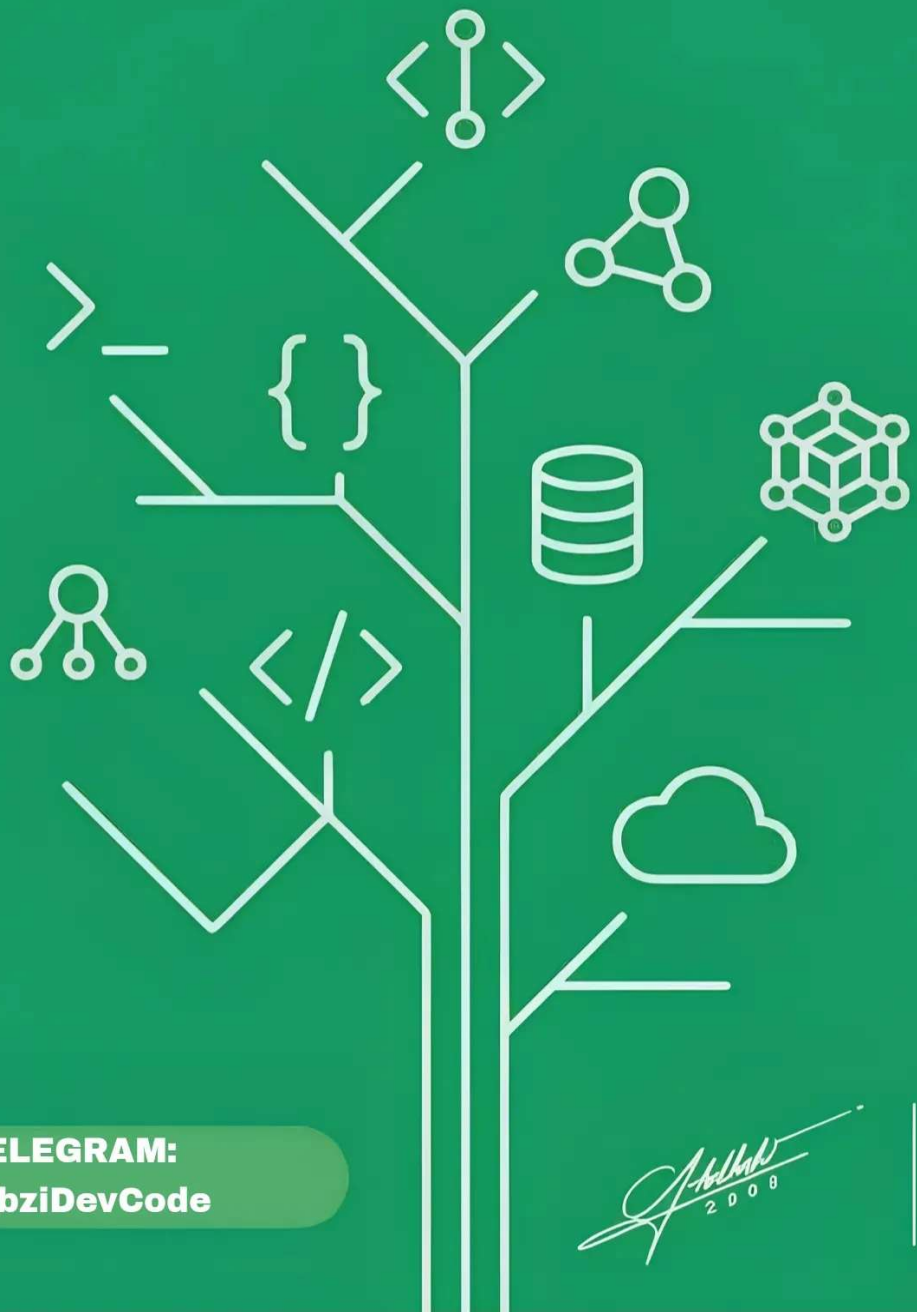
کالبدشکافی تایپ اسکریپت

استراتژی ها، تکنیک ها، بست پرکتیس ها و چیزایی که از تایپ‌اسکریپت نمی‌دونید!



نوشته‌ای از

ابوالفضل سبزمحمدی

TELEGRAM:
@SabziDevCode

سرفصل ها

- مقدمه و هدف کتابچه
- مفاهیم اساسی
- تایپ سیستم (Type System)
- استخراج تایپ (Type Inference)
- طراحی تایپ (Type Design)
- تنظیمات و کانفیگ‌های کاربردی کامپایلر
- منابع

مقدمه و هدف کتابچه

این کتابچه قرار نیست یه مرجع کامل باشه یا همه‌چیز رو از صفر توضیح بده.

فقط چیزاییه که خودم موقع کار باهاشون درگیر شدم، وقت گذاشتم، و آخرش به یه نتیجه‌ی تمیز رسیدم. چیزایی که اگه کسی همون اول کنارم می‌نوشت، شاید چند ساعت یا چند روز وقت و اعصابم رو نجات می‌داد.

پس اگه وسطش به نکته‌ای برخوردی که به‌نظرت می‌شد بهتر گفت — حق داری. این محتوا از تجربه‌ی منه، نه از یه منبع رسمی.

به طور خلاصه هدف این کتابچه و pdf ساده‌ست: اینکه تو مسیر یادگیری، یه قدم کوتاه‌تر از من راه بری!

این کتابچه پیشرفته تایپ اسکریپت، شما را با قدرت type inference، conditional types و mapped types و... آشنا می کند تا کدهایی ایمن، انعطاف پذیر و مقیاس پذیر طراحی کنید. با تسلط بر این مفاهیم، به یک توسعه دهنده حرفه ای تبدیل می شوید که می‌تواند چالش های پروژه های بزرگ را به راحتی مدیریت کند.

مفاهیم اساسی

در این فصل به مفاهیم پایه و اساسی TypeScript پرداخته می شه.

تفاوت های کلیدی با JavaScript، مفاهیم بنیادی مانند Structural Typing، مفهوم Excess Property Checking، تفاوت بین Value Space و Type Space و همچنین مبحث بشدت جذاب تایپ ها مجوعه‌ای از مقادیر هستند به طور مفصل بررسی می‌شن.

تایپ اسکریپت دقیقا چه تفاوتی با جاوا اسکریپت داره؟

تایپ اسکریپت درواقع یه supersetای از جاوا اسکریپت هستش!

Superset (ابرمجموعه) یعنی چیزی که تمام قابلیت‌های یک مجموعه رو دارد، به‌علاوه قابلیت‌های بیشتر. (کلمه ابرمجموعه و مجموعه رو به خاطر بسپارید! بعدا باهاشون کار داریم...*)

JavaScript: زبان اصلی با قابلیت‌های خودش.

TypeScript: تمام قابلیت‌های JavaScript رو دارد، به‌علاوه قابلیت‌هایی مثل استاتیک تایپ‌ها.

به شرطی که برنامه های جاوا اسکریپتی که می‌نویسیم هیچ‌گونه سینتکس اروری نداشته باشند!

و این نکته رو هم توجه داشته باشید که: با وجود اینکه فایل‌های TypeScript پسوند .ts دارن، این به معنی متفاوت بودن کامل TypeScript از JavaScript نیست؛ بلکه TypeScript یک Superset از JavaScript محسوب می‌شه که قابلیت‌هایی مثل Static Typing رو به اون اضافه می‌کنه! (مثلا پروژه های ری اکتی پسوندشون .jsx هست، مگه ما ری اکت رو چیزی جدا از جاوا اسکریپت می‌دونیم؟!)

پس ما می‌تونیم بگیم همه برنامه های جاوا اسکریپتی، تایپ اسکریپتن اما برعکسش درست نیست. چون ts جدا از اینکه همه قابلیت های جی اس رو در خودش داره، سینتکس خاص و تایپ سیستم خودشو داره.

تفاوت js و ts خیلی مهمه، اما من نمیخوام اینجا وارد بحث مقدمات بشم (قبل خوندن این pdf باید تایپ اسکریپت رو به‌خوبی بلد باشید)، شما باید بدونید که تایپ اسکریپت با وجود اینکه Structural Typing هست و خیلی کمک می‌کنه که ما در فاز دولوپ ارور ها رو پیدا و catch بکنیم، ممکنه شما بیاید این زبان رو با خیلی از زبان های دیگه‌ای مثل سی شارپ یا c++ مقایسه کنید؛ که تفاوت های بسیار زیادی بین این‌ها وجود داره!

کاری که تایپ سیستم تایپ اسکریپت انجام میده اینه که بیاد رفتار runtime جاوا اسکریپت رو مدل‌سازی و شبیه‌سازی کنه، یعنی کدی که تولید می‌کنه باید جاوا اسکریپت بتونه ران‌اش بکنه.

حالا ما داخل جی اس یسری operatorها و محاسبات عجیبی داریم:

```
const foo = 1 + "2";

console.log(foo); // output: "12"
```

مشابه کد بالا در یسری زبان‌های برنامه نویسی غیر معتبر و invalid هست.

کد بالا در جاوا اسکریپت و همچنین در تایپ اسکریپت* هردو valid هستند!

اما:

```
console.log(null + 7); // output: The value 'null' cannot be used here.
```

این کد در جاوا اسکریپت valid اما در تایپ اسکریپت invalid هستش!

حالا چرا یسری از عملیات ها در تایپ اسکریپت معتبر و یسری از عملیات ها غیر معتبر هستند؟

این موضوع کاملا بر می‌گرده به طراحی تایپ اسکریپت، یعنی اون مهندسینی که این زبان را ساخته‌اند تصمیم گرفتند هر عملیات چه رفتاری داشته باشه (به اصطاح تایپ‌چکر pass بشه یا نشه 🤦‍♂️)

از Duck Typing تا Structural Typing در TypeScript**

تا اینجا ما چندین بار از اصطلاح Structural Typing استفاده کردیم؛ اما اصلا Structural Typing یعنی چی؟! و ربطش به داک تایپینگ در خود جاوا اسکریپت چیه؟

🦆 Duck Typing در جاوا اسکریپت

در JavaScript، معمولا مهم نیست یک آبجکت از چه کلاسی ساخته شده؛ مهم اینه که چه ویژگی‌ها و رفتارهایی داره. به این رویکرد **Duck Typing** می‌گن.

بذارید ساده‌تر بگم: این کانسپت میگه اگه یه چیزی ظاهرا شبیه duck (اردک) باشه، مثل اردک شنا بکنه، مثل اردک کواک کواک بکنه پس می‌تونیم بگیم اون چیزی که این رفتار ها رو داره احتمالا یه اردکه!

حالا ایده این مفهوم اینه که اگه شما تایپ یه آبجکت رو بشناسید، لزومی نداره که صریحا تایپ اون آبجکت رو تعریف بکنید؛ اگه رفتار و خصوصیاتش مشابه یه تایپی باشه پس ما می‌تونیم بگیم این آبجکت از اون تایپ هم هستش!

```
interface User{
name: string;
}

const user: User = {
name: "ali",
age: 12,
}; //Object literal may only specify known properties, and 'age' does not exist
```

در اینجا ما یک `interface` به نام `User` تعریف کردیم که میگه هر آبجکتی که قرار است با این تایپ سازگار باشد، باید یک ویژگی به نام `name` از نوع `string` داشته باشد.

حالا آبجکت `user` علاوه بر ویژگی `name`، یک ویژگی دیگه به نام `age` هم دارد.

حالا آیا این آبجکت با تایپ `User` سازگار است؟

در نگاه اول شاید فکر کنیم این آبجکت با تایپ `User` **سازگار** نیست؛ چون ویژگی `age` در اینترفیس تعریف نشده است. اما علت خطا این نیست که این دو از نظر ساختاری ناسازگار هستند! علت این است که TypeScript هنگام انتساب مستقیم یک آبجکت تازه، بررسی ویژه ای به نام Excess Property Checking انجام می‌دهد و وجود ویژگی اضافی `age` را گزارش می‌کند.

```ts
interface User {
  name: string;
}

const user = {
  name: "ali",
  age: 12,
};

const anotherUser: User = user;
```

اما این کد بالا مجاز است!

چرا اینجا خطا نداریم؟ چون ابتدا آبجکت را در متغیر `user` ذخیره کرده‌ایم. سپس TypeScript سازگاری ساختاری آن را با `User` بررسی میکند.

نکته: Structural Typing می‌گوید سازگاری تایپ ها بر اساس اعضای مورد نیاز آنها سنجیده می‌شود؛ اما این به معنی نادیده گرفتن همه بررسی های اضافی TypeScript نیست!

Excess Property Checking چیه؟

به طور خیلی ساده یعنی «بررسی ویژگی های اضافی» در TypeScript.

این بررسی کمک می‌کند وقتی یک آبجکت را مستقیم به یک تایپ مشخص نسبت می‌دهی، اگر ویژگی هایی داشته باشد که در آن تایپ تعریف نشده اند، TypeScript به تو هشدار بدهد.

```
interface User {
  name: string;
}

const user: User = {
  name: "ali",
  age: 12,
};
/* Object literal may only specify known properties,
and 'age' does not exist in type 'User'. */
```

چرا؟ چون در `User` فقط ویژگی `name` تعریف شده، اما تو هنگام تعریف مستقیم آبجکت، ویژگی `age` را هم اضافه کرده ای.

پس چرا این مثال خطا نمی‌داد؟

```
interface User {
  name: string;
}
const user = {
  name: "ali",
  age: 12,
};

const anotherUser: User = user;
```

TypeScript در این انتساب، سازگاری ساختاری را بررسی می‌کند:

- `name` وجود دارد.
- نوع `name` برابر `string` است.
- وجود `age` اضافی مانع سازگاری ساختاری نیست.

بنابراین TypeScript در زمان تعریف مستقیم یک آبجکت، سخت گیری بیشتری اعمال میکند تا ویژگی های ناشناخته را به تو گزارش دهد.

پس چرا وقتی آبجکت را در متغیر می‌ریزیم، رفتار متفاوت می‌شد؟

چون در اینجا دو موضوع از هم جدا شده اند:

1. ابتدا آبجکت `user` ساخته شده و TypeScript نوع آن را از روی ساختارش تشخیص داده است.

2. سپس بررسی شده که آیا این آبجکت برای استفاده به عنوان `User` مناسب است یا نه. (در این مرحله، قانون اصلی Structural Typing اعمال میشود: وجود اعضای مورد نیاز کافی است و اعضای اضافی مانع سازگاری نیستند)

پس ارتباط Duck Typing با این موضوع چیست؟

در JavaScript هم همین ایده را در زمان اجرا میبینیم:

```
const user = {
  name: "ali",
  age: 12,
};

function printName(person) {
  console.log(person.name);
}

printName(user); // ali
```

تابع فقط به `name` نیاز دارد. اهمیتی ندارد آبجکت چه ویژگی های دیگری دارد یا از چه کلاسی ساخته شده است.

این ایده Duck Typing است.

TypeScript عمدا بین سازگاری ساختاری و بررسی ویژگی های اضافی (Excess Property Checking) تفاوت می‌گذارد. قانون اصلی می‌گوید ساختار سازگار کافی است؛ اما بررسی اضافه کمک می‌کند اشتباهات احتمالی هنگام نوشتن آبجکت های تازه زودتر پیدا شوند.

Language Serviceها در TypeScript

Language Service در TypeScript سرویسی است که امکانات هوشمند تایپ‌اسکریپت را در ادیتورهایی مثل VS Code فراهم می‌کند؛ مثل تکمیل خودکار کد، نمایش خطاها، رفتن به تعریف یک متغیر و همچنین یسری فیچرهایی برای ریفکتورینگ در اختیارمون قرار می‌دهد.

Value Space و Type Space

در TypeScript دو مفهوم مهمی داریم به نام Value Space و Type Space. برای فهمیدن تفاوتشان، باید بدانیم که بعضی چیزها در کد، مقدار واقعی هستند و بعضی چیزها فقط برای تعریف و بررسی تایپ ها استفاده می‌شوند.

Value Space چیست؟

به فضایی از کد می‌گوییم که مقادیر قابل استفاده در زمان اجرا در آن قرار می‌گیرند؛ مثل متغیرها، توابع و آبجکت ها.

```
const name = "Ali";

function greet() {
  console.log("Hello");
}

greet();
```

در این مثال:

- `name` یک متغیر است که مقدار `"Ali"` را نگه میدارد.
- `greet` یک تابع واقعی است که میتوان آن را اجرا کرد.

این موارد در خروجی نهایی JavaScript حضور دارند.

Type Space چیست؟

به فضایی می‌گوییم که تعریف های مربوط به تایپ‌ها در آن قرار می‌گیرند؛ مثل `type` و `interface`.

که همانطور هم از قبل می‌دانید در runtime حذف می‌شوند!

```ts
interface User {
  name: string;
  age: number;
}

type ID = string | number;
```

در این مثال:

- `User` یک `interface` است.
- `ID` یک Type Alias است.

این دو برای بررسی تایپ ها توسط TypeScript استفاده می‌شوند و بعد از کامپایل به JavaScript، کد های مربوطه‌شان حذف می‌شود!

حالا یه نکته ای*:

مگه ما نگفتیم این 2 فضا، جدا از یکدیگرند؟! اما یک نام می‌تواند نیز در هر دو فضا باشد!

```ts
class User {
  name: string;

  constructor(name: string) {
    this.name = name;
  }
}

type UserType = User;

const user = new User("Ali");
```

اینجا `User` هم به عنوان یک مقدار قابل استفاده در زمان اجرا وجود دارد، چون کلاس `User` می‌تواند نمونه بسازد؛ هم به عنوان یک تایپ قابل استفاده است، چون TypeScript از کلاس برای تعیین نوع نمونه ها استفاده می‌کند.

همچنین بسری از عملگر ها نیز، هم در فضای value space و هم در فضای type space وجود دارند؛ مثل عملگر های typeof و instanceof و... .

حالا بیاید از یه دید جالب‌تری به این موضوع نگاه کنیم:

```
interface Course {}

const react: Course = {}

//1.
type T1 = typeof react; // Course
/*----------------------------------------------------------------------*/
//2.
const runtimeType = typeof react;


console.log(runtimeType); // 'object'
```

در کد بالا، بخش اول Type Space و بخش دوم Value Space است!

و درواقع به همین خاطر هستش که **شما نمی‌توانید مستقیم خود تایپ‌ها رو در تایپ اسکریپت لاگ بندازید**؛ زیرا با هشدار:

" 'NAME' only refers to a type, but is being used as a value here "

روبه‌رو می‌شوید.

تایپ ها را به عنوان مجوعه‌ای از مقادیر در نظر بگیرید!

یادتونه تو کلاس نهم نظریه مجموعه ها رو داشتیم؟ (a اشتراک b - b اجتماع a)

حالا بیاید یکم تایپ اسکریپت رو به نظریه مجموعه‌ها ربط بدیم:

- تایپ never درواقع همان **توهی** هستش (مجموعه‌ای که هیچ مقداری نداره)!
- یا مثلا وقتی که شما تایپ‌های union می‌نویسید، درواقع دارید مجموعه‌ای از عضو ها می‌نویسید!

رابطه مجموعه ها با Union Type در TypeScript خیلی مهمه، چون وقتی تایپ ها را به شکل مجموعه ای از مقادیر ببینی، فهمیدن Union راحت تر میشه.

1. بیا رابطه را با مجموعه ها ببینیم:

```
type A = "ali" | "reza";
type B = "ali" | "reza" | "sara";
```

هر تایپ را مثل یک مجموعه از مقادیر ممکن در نظر بگیر:

مجموعه A --> reza - ali

مجموعه B --> ali - reza - sara

تمام مقادیر A داخل B هستند؛ پس A زیرمجموعه B است.

که یعنی:

$$A \subseteq B$$

2. Union یعنی اجتماع دو مجموعه

در ریاضی، اجتماع دو مجموعه یعنی تمام اعضای هر دو مجموعه را کنار هم قرار بدهیم، بدون تکرار.

در TypeScript هم `|` همین مفهوم را دارد:

```ts
type A = "ali" | "reza";
type B = "sara" | "reza";

type C = A | B;
```

تایپ `C` در نهایت این مقادیر را می پذیرد:

```ts
type C = "ali" | "reza" | "sara";
```

و از از نظر مجموعه ها:

$$C = A \cup B$$

رابطه مجموعه ها با Union Type در TypeScript خیلی مهمه، چون وقتی تایپ ها را به شکل مجموعه ای از مقادیر ببینی، فهمیدن Union راحت تر میشه.

3. نکته مهم: رابطه Union با زیرمجموعه بودن

به این مثال دقت کن:

```
type A = "ali" | "reza";
type B = "ali" | "reza" | "sara";

type C = A | B;
```

چون تمام مقادیر `A` از قبل در `B` وجود دارند، اجتماع آن ها برابر با خود `B` است:

$$A \cup B = B$$

پس `C` عملا همان مقادیر `B` را می پذیرد.

۴. حالا این رابطه در انتساب تایپ ها چه کاربردی دارد؟

```
type Small = "ali" | "reza";
type Big = "ali" | "reza" | "sara";

let big: Big = "sara";
let small: Small;

small = big; // خطا
```

چرا خطا می دهد؟ چون متغیر `big` ممکن است مقدار `"sara"` داشته باشد، ولی `small` فقط `"ali"` یا `"reza"` را می پذیرد.

اما جهت برعکس ولید است:

```ts
let small: Small = "ali";
let big: Big;

big = small; // معتبر
```

چون هر مقداری که `small` بتواند داشته باشد، در مجموعه مقادیر مجاز `big` هم وجود دارد.

پس اگر مجموعه مقادیر یک تایپ کاملا داخل مجموعه مقادیر تایپ دیگر باشد، مقدار تایپ اول را می توان به تایپ دوم نسبت داد.

این یکی از مهم ترین راه ها برای درک رابطه Union Type و سازگاری تایپ ها در TypeScript است.

و intersection typeها یکی از استثناهاتی هستش که با قائده مجموعه ها تضاد داره. (کمی راجبش تفکر کنید!)

حالا قدرت ریاضیات رو درک می‌کنید؟!

بعد هنوزم بیسریا میگن ریاضیات در دنیای واقعی مگه چه کاربردی داره! من تا خود صبح می‌تونم از کوچیکترین مسئله های ریاضی کاربرد هاشو در دنیای واقعی مثال بزنم... ;)

مشکل از شما نیست! از اون معلمیه که اصولی بهتون یاد نداده!!

شاید اگه قبل دوران راهنمایی یا دبیرستان برنامه نویس می‌شدم، ریاضیات رو خیلی بهتر می‌تونستم درک کنم.

تایپ سیستم (Type System) & استخراج تایپ (Type Inference)

در بخش اول این فصل به جزئیات پیشرفته Type System در TypeScript پرداخته می‌شه.

مباحثی مانند تفاوت Type Annotation و Type Assertion و Type Excess Property اجتناب از Object Wrapper Type و بهترین روش های تعریف تایپ برای Function Expressionها بررسی می‌شن.

همچنین موارد کاربردی مانند انتخاب بین interface و type alias، استفاده از ReadOnly برای جلوگیری از Mutation، و محدودیت های استفاده از Index Signature به طور مفصل آموزش داده می‌شن.

از این فصل به بعد قراره مباحث جذاب تایپ اسکریپتی مثل بست پرکتیس ها و روش های اصولی طراحی تایپ گفته بشه.

و در بخش دوم این فصل به مباحث پیشرفته Type Inference و Type Narrowing در TypeScript پرداخته می شه.

موضوعات کلیدی شامل استخراج خودکار تایپ‌ها توسط کامپایلر، تفاوت بین as const و satisfies، و تکنیک های محدود کردن تایپ‌ها (Type Narrowing) می شن.

همچنین مفاهیم مهمی مانند Contextual Typing و Evolving Type بررسی شده و بهترین روش های مدیریت URLها با استفاده از API Specification آموزش داده می‌شه.

سعی کنید تا حد امکان از Type Annotationها به جای Type Assertionها استفاده کنید!

در TypeScript چند قابلیت وجود دارد که به ما اجازه می‌دهند نوع یک مقدار را مشخص کنیم یا به کامپایلر بگوییم که درباره نوع آن اطمینان بیشتری داریم (درواقع به کامپایلر تایپ اسکریپت می‌گوییم من بیشتر از تو می‌دونم! تو کارت نباشه!).

سه مورد از این قابلیت ها **Type Assertion**، **Non-null Assertion** و **Type Annotaion** هستند.

1. Non-null Assertion Operator (`!`)

عملگر `!` به TypeScript می‌گوید که یک مقدار در این نقطه از کد، `null` یا `undefined` نیست؛ حتی اگر کامپایلر نتواند این موضوع را اثبات کند.

```
const btn = document.getElementById('btnSubmit')!;
```

متد `getElementById` ممکن است `null` برگرداند، چون شاید عنصری با این شناسه در صفحه وجود نداشته باشد. TypeScript نیز این احتمال را در نوع خروجی در نظر می‌گیرد.

اما با قرار دادن `!` پس از فراخوانی متد، به کامپایلر میگوییم که فرض کند مقدار برگشتی `null` یا `undefined` نیست.

نکته مهم: این عملگر در زمان اجرا بررسی انجام نمی‌دهد و مقدار `null` را هم تغییر نمی‌دهد؛ پس با عملگر های ! (نقیض) و ? (آپشنال) در خود جاوا اسکریپت تفاوت دارند.

اگر عنصر واقعا وجود نداشته باشد، ممکن است هنگام استفاده از آن با خطای زمان اجرا روبه رو شویم.

2. Type Assertion با `as`

Type Assertion به ما اجازه می‌دهد نوعی را که TypeScript برای یک مقدار در نظر میگیرد، به نوع دیگری تغییر دهیم؛ البته این تغییر فقط در سطح سیستم نوع و تایپ است و مقدار واقعی را تغییر نمی‌دهد.

برای مثال:

```
document.querySelector('#btnSubmit')
  ?.addEventListener('click', e => {
    const btnSubmit = e.currentTarget as HTMLButtonElement;
  });
```

در این مثال، با استفاده از `as HTMLButtonElement` به TypeScript می‌گوییم که `e.currentTarget` را در این بخش از کد از نوع `HTMLButtonElement` در نظر بگیرد.

این کار زمانی مفید است که ما اطلاعات بیشتری درباره مقدار داریم، اما کامپایلر نمی‌تواند آن را به طور کامل تشخیص دهد.

اما Type Assertionها محدودیت های مهمی نیز دارند که اکثر دولوپر ها بطور مستقیم نمی‌دانند*:

TypeScript اجازه نمی‌دهد هر تایپی را بدون محدودیت به هر تایپ دیگری تبدیل کنیم.

برای مثال:

```ts
interface Course {
  id: number;
  title: string;
}

const btn = document.getElementById('btnSubmit')!;

const course = btn as Course;
```

TypeScript خطایی مشابه این نمایش می دهد:

```
Conversion of type 'HTMLElement' to type 'Course'
may be a mistake because neither type sufficiently
overlaps with the other.
```

دلیل این خطا آن است که `HTMLElement` و `Course` از نظر ساختار، ویژگی مشترک کافی ندارند. یک عنصر HTML ویژگی‌هایی مانند `id` و `title` با نوع مورد انتظار اینترفیس `Course` به شکل تعریف شده در آن اینترفیس ندارد.

بخوام به زبان ساده‌تر بگم، درواقع عملگر `!` زیرمجموعه ای از تایپ btn هست، اما Course نمی‌تونه زیرمحموعه‌ای از btn باشه! چون هیچ وجه اشتراکی برای convert شدن با یکدیگر ندارند!!!*

TypeScript این تبدیل را مشکوک تشخیص می‌دهد، چون ممکن است برنامه نویس به اشتباه دو نوع نامرتبط را به هم تبدیل کرده باشد.

اما اگر در مثال قبلی اصرار به کانورت btn به Course را داشته باشید:

اگر واقعا بخواهیم Type Assertion را بین دو نوع نامرتبط انجام دهیم، میتوانیم از یک تایپ واسطه مانند `unknown` استفاده کنیم:

```ts
interface Course {
  id: number;
  title: string;
}

const btn = document.getElementById('btnSubmit')!;

const course = btn as unknown as Course;
```

این روش را **Double Assertion** می‌نامند، چون دو بار از `as` استفاده شده است:

1. ابتدا مقدار را به `unknown` تبدیل میکنیم.
2. سپس آن را به `Course` تبدیل میکنیم.

نوع `unknown` میتواند هر مقداری را در خود جای دهد، اما برخلاف `any`، پیش از استفاده از مقدار به عنوان یک نوع مشخص، باید آن را بررسی کنیم یا یک Assertion مناسب انجام دهیم.

به همین دلیل، تبدیل به `unknown` و سپس به `Course` محدودیت تبدیل مستقیم را دور می‌زند.

رابطه مجموعه‌ها بین Typeها:

برای درک این محدودیت، میتوان نوع ها را مانند مجموعه ای از مقادیر در نظر گرفت.

- اگر مجموعه مقادیر یک نوع کاملا در مجموعه مقادیر نوع دیگر قرار بگیرد، معمولا تبدیل بین آنها برای TypeScript قابل قبول تر است.
- اگر دو نوع هیچ همپوشانی معناداری نداشته باشند، تبدیل مستقیم ممکن است رد شود.
- `unknown` مجموعه بسیار گسترده ای از مقادیر را میپذیرد؛ به همین دلیل میتواند نقش یک نوع واسطه را داشته باشد.

در اینجا باید به یک نکته دقت کنیم: TypeScript صرفا براساس وجود یک ویژگی مشترک تصمیم نمی‌گیرد. سازگاری ساختاری، نوع ویژگی ها و قواعد سیستم تایپ نیز در تصمیم آن نقش دارند.

پس درواقع هر تایپی می‌تونه زیرمجموعه و ساب‌تایپ unknown باشه!

و برعکس این جمله هم حتی درسته! unknown می‌تونه ساب‌تایپ هر تایپ دیگه‌ای هم باشه...

نکته: Type Assertion به‌طور کلی Type Checking رو غیرفعال نمی‌کنه؛ بلکه در همون عبارت، بعضی بررسی‌های نوع رو دور می‌زنه. به همین دلیل استفاده نادرست از اون می‌تونه خطاهای نوعی رو پنهان کنه.

از Object Wrapper Type استفاده نکنید

در TypeScript و JavaScript، برای تعریف نوع مقادیر اولیه (Primitive Types) همیشه از تایپ‌های کوچکی مانند `boolean`، `number`، `string` و `bigint` استفاده کنید، نه تایپ‌های شی‌گرا و بزرگ مانند `Boolean`، `Number`، `String` و `BigInt`.

این تایپ‌های بزرگ، Object Wrapper Type نام دارند و نماینده شیءهایی هستند که دور مقادیر اولیه قرار می‌گیرند. برای مثال:

```
const name: string = "Abolfazl"; // ✅
const name: String = new String("Abolfazl"); // ❌
```

تفاوت این دو فقط در ظاهر نیست. مقدار اولیه و شیء متناظر آن رفتار یکسانی ندارند:

```
const a = "hello";
const b = new String("hello");

console.log(typeof a); // "string"
console.log(typeof b); // "object"

console.log(a === b); // false
```

استفاده از Wrapper Typeها می‌تواند باعث ناسازگاری در مقایسه‌ها، بررسی تایپ و انتقال داده‌ها شود. همچنین در TypeScript، تایپ‌های بزرگ معمولا مقادیر اولیه را هم می‌پذیرند، اما این موضوع دلیلی برای استفاده از آن‌ها نیست.

در کل برای تایپ‌گذاری از `number`، `string` و `boolean` استفاده کنید. حروف بزرگ را فقط در مواردی به کار ببرید که واقعا به خود سازنده یا شیء متناظر نیاز دارید، نه برای تعریف تایپ معمولی یک مقدار.

مفهوم مهم Excess Property Checking

Excess Property Checking به معنی «بررسی ویژگی‌های اضافی» است. در TypeScript، زمانی که یک Object Literal را مستقیما به متغیری با یک تایپ مشخص اختصاص می‌دهید یا به تابعی ارسال می‌کنید، TypeScript ویژگی‌هایی را که در نوع مقصد تعریف نشده‌اند بررسی می‌کند و معمولا برای آن‌ها خطا می‌دهد.

```ts
type User = {
  name: string;
  age: number;
};

const user: User = {
  name: "Abolfazl",
  age: 18,
  email: "test@example.com", // ❌ خطا
};
```

ویژگی `email` در نوع `User` تعریف نشده است؛ بنابراین TypeScript آن را به‌عنوان یک ویژگی اضافی شناسایی می‌کند.

اما نکته مهمی که وجود داره اینه که TypeScript بررسی ویژگی‌های اضافی را در همه شرایط انجام نمی‌دهد. برای مثال، اگر ابتدا شیء را در یک متغیر ذخیره کنید، ممکن است همان انتساب معتبر باشد:

```ts
const data = {
  name: "Abolfazl",
  age: 18,
  email: "test@example.com",
};

const user: User = data; // ✅ بدون خطا
```

دلیل این رفتار، سیستم نوع‌دهی ساختاری (Structural Typing) در TypeScript است. متغیر `data` تمام ویژگی‌های موردنیاز `User` را دارد؛ وجود ویژگی اضافی `email` به‌تنهایی مانع انتساب نمی‌شود.

Excess Property Checking مکانیزمی برای شناسایی اشتباهات احتمالی هنگام تعریف مستقیم Object Literal است، نه قانونی که همیشه وجود ویژگی‌های اضافی را ممنوع کند.

تفاوت Excess Property Checking و Structural Typing در TypeScript

- Structural Typing (تایپ‌دهی ساختاری): سازگاری دو تایپ بر اساس ساختار و ویژگی‌های آن‌ها تعیین می‌شود، نه اسمشان.
- Excess Property Checking (بررسی پراپرتی‌های اضافی): بررسی اضافه‌ای است که TypeScript هنگام استفاده مستقیم از Object Literal انجام می‌دهد تا پراپرتی‌های احتمالا اشتباه را شناسایی کند.

```ts
type User = {
  name: string;
};

const data = {
  name: "Abolfazl",
  age: 18,
};

const user: User = data; // ✅ Structural Typing
```

چون `data` پراپرتی `name` را دارد، انتساب معتبر است؛ پراپرتی اضافی `age` مانع آن نمی‌شود.

اما:

```ts
const user: User = {
  name: "Abolfazl",
  age: 18, // ❌ Excess Property Checking
};
```

اینجا چون Object Literal را مستقیم به `User` انتساب داده‌ایم، TypeScript درباره `age` خطا می‌دهد.

پس برای خلاصه: Structural Typing قانون اصلی سازگاری تایپ‌هاست؛ Excess Property Checking یک بررسی اضافه برای Object Literalهاست، نه جایگزین آن قانون.

برای افزایش قابلیت استفاده مجدد برروی کل Function Expression تایپ‌ها را تعریف کنید!

وقتی یک Function Expression را در چند جای پروژه استفاده می‌کنید، بهتر است تایپ تابع را جداگانه تعریف کنید و سپس آن را به تابع اختصاص دهید. این کار باعث می‌شود امضای تابع (Function Signature) یکپارچه باشد و بتوانید همان تایپ را برای توابع مشابه نیز به کار ببرید.

که دیگه مجبور نباشید برای هر تابع مشابه، برای هر پارام هایش و خروجی‌اش دوباره تایپ تعریف کنید! (DRY)

```
type Add = (a: number, b: number) ⇒ number;

const add: Add = (a, b) ⇒ a + b;
const sum: Add = (a, b) ⇒ a + b + 1;
```

در این مثال، تایپ `Add` نوع پارامترها و مقدار بازگشتی تابع را مشخص می‌کند. TypeScript نیز نوع پارامترهای `a` و `b` را به‌صورت خودکار استنباط می‌کند.

این روش در پروژه‌های بزرگ، به‌ویژه برای Callbackها و توابعی که امضای مشترکی دارند، از تکرار تایپ‌ها جلوگیری می‌کند و تغییرات آینده را ساده‌تر می‌سازد.

نکته: این قانون به معنی تعریف تایپ جداگانه برای تمام توابع نیست؛ زمانی از آن استفاده کنید که تایپ تابع قابلیت استفاده مجدد دارد یا می‌خواهید قرارداد مشخصی برای آن داشته باشید.

تا حد امکان از Index Signature استفاده نکنید!

در TypeScript، از تعریف Index Signature برای آبجکت‌هایی که ساختار مشخصی دارند خودداری کنید. به‌جای آن، پراپرتی‌ها را به‌صورت صریح تعریف کنید تا تایپ‌ها دقیق‌تر باشند و TypeScript بتواند اشتباهات بیشتری را شناسایی کند.

برای مثال، تعریف زیر اجازه می‌دهد هر کلید رشته‌ای با هر مقداری به آبجکت اضافه شود:

```
type User = {
  [props: string]: unknown;
};
```

در این حالت، TypeScript محدودیت مشخصی برای نام پراپرتی‌ها ندارد. در مقابل، تعریف دقیق‌تر ساختار آبجکت را مشخص می‌کند:

```
type User = {
  name: string;
  age: number;
};
```

اکنون TypeScript می‌تواند مقادیر نامعتبر را تشخیص دهد و از دسترسی به پراپرتی‌هایی که در تایپ تعریف نشده‌اند، در شرایط مناسب جلوگیری کند.

البته Index Signature همیشه اشتباه نیست. زمانی که واقعا با کلیدهای داینامیک کار می‌کنید، مانند یک دیکشنری یا آبجکت تنظیمات با کلیدهای متغیر، استفاده از آن منطقی است. همچنین `unknown` نسبت به `any` انتخاب امن‌تری است، زیرا پیش از استفاده از مقدار باید تایپ آن بررسی شود.

یه قاعده کلی رو درنظر داشته باشید: اگر ساختار آبجکت مشخص است، پراپرتی‌ها را صریح تعریف کنید؛ فقط زمانی از Index Signature استفاده کنید که به کلیدهای داینامیک نیاز دارید.

Mutation؛ عامل بسیاری از خطاها

Mutation به معنی تغییر دادن وضعیت یا محتوای یک مقدار پس از ایجاد آن است. برای مثال، اضافه کردن یک عضو به آرایه، حذف یک پراپرتی از آبجکت یا تغییر مقدار یکی از پراپرتی‌های آن، نوعی Mutation محسوب می‌شود.

Mutation همیشه اشتباه نیست، اما در پروژه‌های بزرگ می‌تواند باعث خطاهایی شود که پیدا کردن آن‌ها دشوار است؛ به‌ویژه زمانی که چند بخش از برنامه به یک مرجع مشترک دسترسی دارند و تغییر یک بخش، رفتار بخش‌های دیگر را نیز تحت تاثیر قرار می‌دهد.

در TypeScript، می‌توان از `readonly` برای محدود کردن تغییرات و جلوگیری از Mutationهای ناخواسته استفاده کرد.

```
const a: number[] = [1, 2, 3];

const b: readonly number[] = a;

const c: number[] = b; // خطا
```

در این مثال، `b` به همان آرایه‌ای اشاره می‌کند که `a` به آن اشاره دارد؛ آرایه جدیدی ساخته نشده است. با این حال، تایپ `readonly number[]` اجازه نمی‌دهد از طریق `b` اعضای آرایه را تغییر دهیم.

انتساب `b` به `c` نیز خطا می‌دهد، زیرا `b` فقط خواندنی است، اما `c` اجازه تغییر اعضای آرایه را می‌دهد. اگر چنین انتسابی مجاز بود، می‌توانستیم از طریق `c` آرایه‌ای را تغییر دهیم که از طریق `b` فقط خواندنی در نظر گرفته شده است.

```
a.push(4); // مجاز
b.push(4); // خطا
```

نکته مهم: `readonly` آرایه را در زمان اجرا غیرقابل تغییر نمی‌کند و جلوی تغییر آن از طریق مرجع دیگری مانند `a` را نمی‌گیرد.

این ویژگی یک محدودیت در سطح تایپ‌گذاری TypeScript است.

هرجا قرار نیست تابع یا بخشی از برنامه داده‌ای را تغییر دهد، استفاده از تایپ‌های `readonly` می‌تواند قرارداد کد را روشن‌تر کند و از تغییرات ناخواسته جلوگیری کند.

اما این رو در نظر داشته باشید که `readonly` deepReadonly نمی‌کنه! بلکه shallow هستش!!

بنابراین پیشنهاد می‌شود تا حد امکان از as const استفاده کنید!*

تا حد امکان از Type Annotation به‌جای Type Assertion استفاده کنید!

در **Type Annotation**، TypeScript برای مشخص کردن تایپ یک متغیر یا پارامتر استفاده می‌شود؛ در مقابل، **Type Assertion** به TypeScript اعلام می‌کند که یک مقدار را به‌عنوان تایپ مشخصی در نظر بگیرد.

Type Assertion بررسی تایپ را به‌طور کامل غیرفعال نمی‌کند، اما می‌تواند برخی بررسی‌های استنباطی را دور بزند و باعث شود TypeScript فرضی را بپذیرد که لزوما در زمان اجرا درست نیست.

```
// Type Annotation
const name: string = "Abolfazl";

// Type Assertion
const name = "Abolfazl" as string;
```

در مثال بالا، استفاده از `as string` ضروری نیست؛ زیرا TypeScript خودش از روی مقدار اولیه، تایپ `string` را استنباط می‌کند. البته در بسری شرایط ممکنه نیاز داشته باشید متغیر رو هم بصورت const تعریف کنید و هم as const بهش بدید؛ مثلا مواقعی که از آبجکت‌ها استفاده می‌کنید.

نمونه‌ای خطرناک‌تر:

```
const value = "123" as unknown as number;
console.log(value.toFixed()); // خطای زمان اجرا
```

با وجود Assertion، مقدار همچنان یک رشته است و به عدد تبدیل نشده است. بنابراین، Type Assertion برخلاف تبدیل واقعی مقدار، تضمین نمی‌کند که مقدار در زمان اجرا با تایپ اعلام‌شده مطابقت داشته باشد.

ابتدا اجازه دهید TypeScript تایپ را استنباط کند و در صورت نیاز از Type Annotation استفاده کنید.

قاعده: ابتدا اجازه دهید TypeScript تایپ را استنباط کند و در صورت نیاز از Type Annotation استفاده کنید. Type Assertion را فقط زمانی به کار ببرید که اطلاعات کافی درباره تایپ مقدار دارید و TypeScript نمی‌تواند آن را به‌درستی استنباط کند. (درواقع انگار شما صریحا میگید آقای/خانم تایپ اسکریپت من از شما بیشتر می‌دونم! تو کارت نباشه من دولوپر چه کار دارم می‌کنم.)

از `number` در تعریف Index Signature استفاده نکنید، مگر در صورت نیاز واقعی

یکم قبل‌تر گفتیم از ایندکس سیگنچر ها استفاده نکنید، اما اگه کردید به این نکته هم توجه داشته باشید:

در TypeScript، استفاده از `number` به‌عنوان کلید در Index Signature همیشه اشتباه نیست، اما باید بدانید که کلیدهای عددی در JavaScript رفتار خاصی دارند. هنگام دسترسی به پراپرتی‌های یک آبجکت معمولی، کلیدهای عددی به رشته تبدیل می‌شوند؛ برای مثال، `obj[1]` و `obj["1"]` به یک پراپرتی اشاره می‌کنند.

```
type Dictionary = {
  [key: number]: string;
};
const data: Dictionary = {
  1: "one",
  2: "two",
};

console.log(data[1]); // "one"
console.log(data["1"]); // "one"
```

این رفتار ممکن است باعث شود تصور کنیم کلیدهای عددی و رشته‌ای کاملا مستقل هستند، در حالی که در آبجکت‌های معمولی چنین نیست. همچنین Index Signature عددی به این معنا نیست که آبجکت فقط می‌تواند کلیدهای عددی داشته باشد.

اگر هدف شما ذخیره و دسترسی به داده‌ها بر اساس اندیس‌های عددی است، معمولا آرایه‌ها انتخاب مناسب‌تری هستند. اگر به یک مجموعه از کلیدهای داینامیک نیاز دارید، می‌توانید از `Record` یا Index Signature رشته‌ای استفاده کنید؛ انتخاب درست به ساختار داده و نیاز برنامه بستگی دارد.

صرفا به این دلیل که کلیدهای شما ظاهرا عددی هستند، از Index Signature عددی استفاده نکنید. ابتدا مشخص کنید داده واقعا آرایه است، یک دیکشنری با کلیدهای عددی است یا آبجکتی با کلیدهای رشته‌ای.

برای تایپ های مختلف متغیرهای مختلف تعریف کنید

در TypeScript، بهتر است از تغییر تایپ یک متغیر در طول اجرای برنامه خودداری کنید. هر متغیر را با توجه به هدف و نوع داده‌ای که نگهداری می‌کند تعریف کنید تا کد خواناتر باشد و TypeScript بتواند خطاهای بیشتری را شناسایی کند.

برای مثال، به‌جای استفاده مجدد از یک متغیر برای نگهداری مقادیر با تایپ‌های متفاوت، متغیرهای جداگانه تعریف کنید:

```
let value: string = "123";

value = 123; // خطا
```

در این مثال، متغیر `value` از ابتدا با تایپ `string` تعریف شده است؛ بنابراین انتساب یک مقدار عددی به آن مجاز نیست.

اگر به هر دو مقدار نیاز دارید، متغیرهای جداگانه تعریف کنید:

```
const textValue = "123";
const numberValue = 123;
```

این روش باعث می‌شود هدف هر متغیر مشخص باشد و از تبدیل‌های ناخواسته، سردرگمی در خواندن کد و خطاهای احتمالی جلوگیری شود.

البته اگر یک متغیر واقعا باید در طول برنامه مقادیر متفاوتی از چند تایپ مشخص بپذیرد، می‌توانید از Union Type استفاده کنید:

```
let value: string | number = "123";
value = 123; // معتبر
```

برای داده‌هایی با کاربردهای متفاوت، متغیرهای جداگانه تعریف کنید. فقط زمانی از یک متغیر با Union Type استفاده کنید که تغییر تایپ بخشی از منطق واقعی برنامه باشد.

```ts
let courseId: string | number = "CLB450";

if (typeof courseId === "string") {
  const s = courseId;
  // تایپ s برابر string است
}

courseId = 54120;

if (typeof courseId === "number") {
  // این شرط برقرار است
  console.log(typeof courseId); // "number"
}
```

اینجا نکته اصلی اینه که TypeScript در هر بخش از کد، تایپ متغیر رو بر اساس مقداردهی‌ها و کنترل جریان تشخیص می‌ده؛ اما `typeof` در زمان اجرا مقدار واقعی متغیر رو بررسی می‌کنه.

مثلا همین کد بالا رو میشه با 2تا متغیر جدا هندل کرد که یکی آیدی کورس باشه و دیگری سریال نامبر یا ..اش باشه!!

*****کار کردن با سینگل تایپ ها به مراتب خیلی راحت تر از یونیون تایپ ها هستش!*****

چون وقتی یک متغیر فقط یک تایپ مشخص داره، لازم نیست حالت‌های مختلفش رو بررسی کنی:

```ts
let courseId: string = "CLB450";

const s = courseId.toUpperCase(); // بدون بررسی اضافی
```

اما با Union Type:

```ts
let courseId: string | number = "CLB450";
courseId.toUpperCase(); // چون خطا! courseId ممکنه عدد باشه
```

باید اول تایپ رو محدود کنی:

```
if (typeof courseId === "string") {
  courseId.toUpperCase(); // حالا امنه
}
```

اما نکته مهم: این به معنی بد بودن Union Type نیست.

- Single Type: وقتی مقدار فقط یک تایپ مشخص داره، انتخاب ساده‌تر و مناسبیه.
- Union Type: وقتی متغیر واقعا می‌تونه چند تایپ داشته باشه، مدل‌سازی دقیق‌تری ارائه می‌ده؛ فقط باید حالت‌های مختلفش رو مدیریت کنی.

بهترین رویکرد اینه که تا جای ممکن، تایپ‌ها رو دقیق و محدود تعریف کنی؛ نه اینکه صرفا برای راحت‌تر شدن کد، از Union Type اجتناب کنی.

از Union Typeها کمتر استفاده کنید!

تا جای ممکن، برای متغیرها تایپ مشخص و دقیقی تعریف کنید و فقط زمانی از `Union Type` استفاده کنید که یک مقدار واقعا بتواند چند تایپ متفاوت داشته باشد. استفاده بیش از حد از Union Typeها می‌تواند نیاز به بررسی تایپ و مدیریت حالت‌های مختلف را افزایش دهد و خوانایی کد را کاهش دهد.

نکته: هدف، حذف Union Typeها نیست؛ بلکه استفاده از آن‌ها فقط در مواقع ضروری و متناسب با منطق برنامه است.

اگه فانکشنت داره 2تا تایپ مختلف ریتورن می‌کنه، قشنگ داره سیگنال میده که یجای کار می‌لنگه! کدی کلینه که هی ایف الس نداشته باشه!

Context عامل مهمی دیگر در تشخیص تایپ‌ها

یادتون باشه که TypeScript برای تشخیص تایپ یک عبارت، فقط به خود آن عبارت نگاه نمی‌کند؛ بلکه Context یا محل استفاده از آن را هم در نظر می‌گیرد. برای مثال، هنگام ارسال یک آرایه به تابعی که ورودی آن یک Tuple با سه مقدار از تایپ `number` است، TypeScript می‌تواند آرایه را متناسب با تایپ مورد انتظار بررسی کند.

با این حال، وقتی مقدار را ابتدا در یک متغیر جداگانه ذخیره می‌کنید، ممکن است TypeScript تایپ آن را به شکل گسترده‌تری استنباط کند. برای حفظ تایپ دقیق‌تر، می‌توانید از **Type Annotation**، عبارت `as const` یا تایپ مناسب دیگری استفاده کنید.

نکته: Context همیشه باعث استنباط تایپ دقیق‌تر نمی‌شود؛ نتیجه به محل استفاده و قواعد استنباط تایپ در TypeScript بستگی دارد.

بیاید یسری مثال های جالبی باهم ببینیم:

1. Literal Union

```ts
type ButtonType = "regular" | "icon" | "circle";

declare function renderButton(type: ButtonType): void;

renderButton("icon"); // معتبر

let iconButton = "icon";
renderButton(iconButton); // خطا
```

`iconButton` به صورت `string` استنباط می‌شود، نه `ButtonType`. چون متغیر `let` قابلیت تغییر مقدار دارد، TypeScript مقدار آن را به تایپ گسترده‌تری تعمیم می‌دهد.

2. Tuple

```
declare function setColor(color: [number, number, number]): void;

setColor([255, 0, 0]); // معتبر

const redColor = [255, 0, 0];
setColor(redColor); // خطا
```

آرایه مستقیم در محل فراخوانی با Context تایپ مورد انتظار بررسی می‌شود؛ اما `redColor` به صورت `[]number` استنباط می‌شود، نه Tuple سه‌عضوی.

3. استفاده از `as const`

```
declare function setColor(
  color: readonly [number, number, number]
): void;

const redColor = [255, 0, 0] as const;

setColor(redColor); // معتبر
```

عبارت `as const` باعث می‌شود آرایه به صورت یک Tuple فقط‌خواندنی با مقادیر Literal استنباط شود. چون پارامتر تابع هم `readonly` است، این Tuple قابل قبول خواهد بود.

نکته‌های مهم درباره استنباط تایپ در TypeScript

1. تا جای ممکن به Type Inference اعتماد کنید

TypeScript می‌تواند در بسیاری از مواقع تایپ متغیرها را از روی مقدار اولیه، مقدارهای بعدی و Context تشخیص دهد. بنابراین، لازم نیست برای هر متغیر حتما Type Annotation بنویسید.

```
const courseName = "TypeScript"; // string
const courseCount = 12; // number
```

قانون کلی بخوام براش بگم: زمانی که TypeScript می‌تواند تایپ را به‌درستی و به‌شکل واضح تشخیص دهد، معمولا نیازی به نوشتن Type Annotation نیست.

2. همیشه Type Annotation را حذف نکنید

حذف Type Annotation لزوما کد را خواناتر نمی‌کند. گاهی مشخص کردن تایپ، قصد برنامه‌نویس را واضح‌تر می‌کند یا جلوی استنباط تایپ نامناسب را می‌گیرد.

```
const courseName: string = "TypeScript";
```

در این مثال، Type Annotation ضروری نیست؛ اما در مواردی مانند ورودی تابع، متغیرهای دارای چند حالت یا آرایه‌های خالی، مشخص کردن تایپ می‌تواند مفید باشد.

فقط برای کاهش تعداد Type Annotationها، تایپ‌گذاری صریح و مفید را حذف نکنید.

3. با Evolving Type آشنا باشید

در بعضی موقعیت‌ها، TypeScript می‌تواند تایپ یک متغیر را با توجه به مقداردهی‌های بعدی تکامل دهد. این رفتار را Evolving Type می‌نامند.

```
let value;

value = "TypeScript";
value = 123;
```

در شرایط مناسب، TypeScript می‌تواند استفاده‌های مختلف از متغیر را بررسی کند و تایپ آن را بر اساس مقادیری که به آن اختصاص داده شده‌اند، تکامل دهد.

این رفتار برای بعضی متغیرهایی که ابتدا بدون مقدار مشخص تعریف می‌شوند، کاربرد دارد؛ اما نباید فرض کنید که همه متغیرهای بدون Type Annotation به همین شکل رفتار می‌کنند.

4. مراقب آرایه‌های خالی باشید

آرایه خالی همیشه تایپ نهایی مناسبی برای TypeScript فراهم نمی‌کند. بسته به Context و تنظیمات کامپایلر، ممکن است به خطای تایپ یا استنباطی نامناسب منجر شود.

```
const numbers: number[] = [];

numbers.push(1);
numbers.push(2);
```

در این مثال، مشخص کردن `number[]` هم هدف کد را روشن می‌کند و هم از اضافه شدن مقدارهایی با تایپ ناسازگار جلوگیری می‌کند.

قانون: وقتی آرایه در ابتدا خالی است و تایپ عناصر آن از Context قابل تشخیص نیست، تایپ آن را صریح مشخص کنید.

5. برای Union Typeها، تایپ را بی‌دلیل گسترده نکنید

اگر یک متغیر فقط قرار است مقدارهایی از یک تایپ مشخص داشته باشد، همان تایپ را حفظ کنید. Union Type را زمانی به کار ببرید که چند تایپ واقعا بخشی از منطق برنامه باشند.

```
let courseId: string = "CLB450";
```

اگر `courseId` واقعا می‌تواند هم `string` و هم `number` باشد، تعریف `string | number` مناسب است؛ اما در غیر این صورت، تایپ محدودتر معمولا استفاده از متغیر را ساده‌تر می‌کند.

6. برای داده‌های خارجی، تایپ را صرفا حدس نزنید

داده‌های دریافتی از API، فرم‌ها و منابع خارجی لزوما با تایپی که انتظار داریم مطابقت ندارند. برای نمونه، `response.json()` به‌تنهایی صحت ساختار داده را تضمین نمی‌کند.

```
const response = await fetch(url);
const data: unknown = await response.json();
```

در این حالت باید داده را بررسی و اعتبارسنجی کنید یا از یک روش معتبر برای اعتبارسنجی ساختار آن استفاده کنید. صرفا نوشتن `as Course` داده را در زمان اجرا اعتبارسنجی نمی‌کند.

7. برای تایپ‌های وابسته به DOM از Narrowing استفاده کنید

بسیاری از متدهای DOM ممکن است مقداری از نوع `null` برگردانند. قبل از دسترسی به پراپرتی‌ها یا متدهای عنصر، وجود آن را بررسی کنید.

```
const progressElement = document.querySelector(".progress");

if (progressElement) {
  progressElement.classList.add("progress-bar");
}
```

TypeScript درون شرط می‌فهمد که `progressElement` دیگر `null` نیست؛ بنابراین می‌توانید با اطمینان بیشتری از آن استفاده کنید.

برای بررسی تایپ واقعی یک شی نیز می‌توانید از `instanceof` استفاده کنید:

```
if (progressElement instanceof HTMLElement) {
  progressElement.classList.add("progress-bar");
}
```

نکته: `instanceof` برای بررسی نمونه بودن یک شی از یک کلاس یا سازنده مناسب است؛ اما جایگزین بررسی `null` در تمام موقعیت‌ها نیست.

از Discriminated Union استفاده کنید!

وقتی چند تایپ ساختارهای متفاوتی دارند، می‌توانید یک پراپرتی مشترک با مقادیر Literal متفاوت تعریف کنید تا TypeScript بتواند تایپ هر حالت را تشخیص دهد.

```
type Success = {
  status: "success";
  data: string;
};

type Error = {
  status: "error";
  message: string;
};

type Result = Success | Error;

function handleResult(result: Result) {
  if (result.status === "success") {
    console.log(result.data);
  } else {
    console.log(result.message);
  }
}
```

در این مثال، `status` پراپرتی متمایزکننده (Discriminant) است. با بررسی مقدار آن، TypeScript تایپ `result` را به حالت مربوط محدود می‌کند و دسترسی به پراپرتی‌های اختصاصی هر حالت را ممکن می‌سازد.

زمانی که یک Union Type شامل چند حالت با ساختارهای متفاوت است، استفاده از Discriminated Union می‌تواند تایپ‌چکینگ را دقیق‌تر و کد را خواناتر کند.

چه زمانی از `as const` استفاده کنیم؟

از `as const` زمانی استفاده کنید که می‌خواهید مقادیر Literal حفظ شوند و ساختار آبجکت یا آرایه به شکل عمیق `readonly` شود.

```
const appConfig = {
  theme: "dark",
  maxUsers: 100,
  features: {
    notifications: true,
    chat: false,
  },
  supportedLanguages: ["en", "fr", "es"],
} as const;
```

در این مثال:

- `theme` تایپ `"dark"` دارد، نه `string`.
- `maxUsers` تایپ `100` دارد، نه `number`.
- پراپرتی‌های آبجکت به شکل `readonly` هستند.
- آرایه `supportedLanguages` به یک Tuple فقط‌خواندنی تبدیل می‌شود.

سناریوهای مناسب:

- تعریف تنظیمات ثابت برنامه
- تعریف مجموعه مقادیر Literal برای وضعیت‌ها و گزینه‌ها
- حفظ تایپ دقیق آرایه‌ها به شکل Tuple
- جلوگیری از تغییر ناخواسته مقادیر از طریق همان مرجع در زمان توسعه

نکته: `as const` داده را در زمان اجرا فریز نمی‌کند؛ فقط محدودیت‌های تایپی ایجاد می‌کند.

پس چه زمانی از `satisfies` استفاده کنیم؟

از `satisfies` زمانی استفاده کنید که می‌خواهید مطمئن شوید یک عبارت با یک تایپ مشخص سازگار است، اما تایپ استنباط‌شده خود عبارت را نیز حفظ کنید.

```
type UserSettings = {
  username: string;
  email: string;
  isDarkMode: boolean;
  code: number;
  preferredLanguages: string[];
  notifications: {
    email: boolean;
    sms: boolean;
    push: boolean;
  };
};

const userSettings = {
  username: "john_doe",
  email: "john.doe@example.com",
  isDarkMode: true,
  code: 12345,
  preferredLanguages: ["en", "fr"],
  notifications: {
    email: true,
    sms: false,
    push: true,
  },
} satisfies UserSettings;

userSettings.email = "jd@example.com"; // معتبر
userSettings.isDarkMode = false; // معتبر
```

سناریوهای مناسب:

-

از `satisfies` زمانی استفاده کنید که می‌خواهید مطمئن شوید یک عبارت با یک تایپ مشخص سازگار است، اما تایپ استنباط‌شده خود عبارت را نیز حفظ کنید.

```
type UserSettings = {
  username: string;
  email: string;
  isDarkMode: boolean;
  code: number;
  preferredLanguages: string[];
  notifications: {
    email: boolean;
    sms: boolean;
    push: boolean;
  };
};

const userSettings = {
  username: "john_doe",
  email: "john.doe@example.com",
  isDarkMode: true,
  code: 12345,
  preferredLanguages: ["en", "fr"],
  notifications: {
    email: true,
    sms: false,
    push: true,
  },
} satisfies UserSettings;

userSettings.email = "jd@example.com"; // معتبر
userSettings.isDarkMode = false; // معتبر
```

سناریوهای مناسب:

- تعریف تنظیمات با بررسی سازگاری تایپ
- بررسی کامل بودن پراپرتی‌های مورد انتظار و تشخیص پراپرتی‌های اضافی در Object Literal
- حفظ تایپ استنباط‌شده برای استفاده‌های بعدی

- جلوگیری از تبدیل غیرضروری تایپ دقیق عبارت به یک تایپ کلی‌تر
- حفظ تایپ استنباط‌شده برای استفاده‌های بعدی

ترکیب هر دو: اگر هم تایپ دقیق و `readonly` می‌خواهید و هم بررسی سازگاری با یک تایپ مشخص، می‌توانید از هر دو استفاده کنید:

```
const appConfig = {
  theme: "dark",
  maxUsers: 100,
} as const satisfies {
  theme: "dark" | "light";
  maxUsers: number;
};
```

چرا در مثال `satisfies` نمی‌توان `isDarkMode` را به `false` تغییر داد؟

اگر `userSettings` به این شکل تعریف شود:

```
const userSettings = {
  isDarkMode: true,
} satisfies { isDarkMode: boolean };
```

در این موقعیت، تایپ `isDarkMode` ممکن است به صورت `true` استنباط شود؛ چون `satisfies` لزوما آن را به `boolean` تبدیل نمی‌کند. بنابراین انتساب `false` خطا می‌دهد.

اگر می‌خواهید مقدار تغییر کند، تایپ را صریح تعریف کنید:

```ts
const userSettings: UserSettings = {
  // ...
};
```

یا تایپ پراپرتی را به شکلی تعریف کنید که مقدارهای مورد نیاز را بپذیرد. یادتان باشد `satisfies` تایپ استنباط‌شده را حفظ می‌کند، اما Context همچنان می‌تواند روی استنباط Literalها اثر بگذارد.

چه زمانی از Interface و چه زمانی از Type Alias استفاده کنیم؟

Interface یا Type Alias؟

هر دو برای تعریف ساختار آبجکت‌ها مناسب‌اند؛ اما `type` قابلیت‌های بیشتری برای ترکیب تایپ‌ها دارد.

- **Interface:** برای تعریف ساختار آبجکت‌ها، قرارداد کلاس‌ها و گسترش ساختارهای موجود با `extends` مناسب است.
- **Type Alias:** برای تعریف Intersection Type، Tuple، Union Type، تایپ‌های شرطی و Mapped Type کاربرد دارد. البته برای تعریف ساختار آبجکت‌ها نیز می‌توان از آن استفاده کرد.

```
interface User {
  id: number;
  name: string;
}

type Status = "loading" | "success" | "error";
type ID = string | number;
```

برای ساختارهای ساده آبجکت می‌توانید از `interface` و برای تایپ‌های ترکیبی و Literal Unionها از `type` استفاده کنید. این یک راهنمای کاربردی است، نه یک قانون اجباری؛ هر دو ابزار در بسیاری از سناریوها قابل استفاده‌اند.

مثلا برای entity های بیزینس‌تون یا modelها و داده‌هایی که استراکچر دارن خوب مشخصه که از اینترفیس باید استفاده کنید!*

تا جای ممکن از Type Inference استفاده کنید!

این موضوع دولوپر ها رو به 2دسته تبدیل می‌کنه! یسریا میگن برای خوانایی همیشه‌ی خدا تایپ‌ها رو صریحا مشخص کنید، اما یسری ها از جمله خودم بر این باوریم که بهتره بذاریم خود تایپ اسکریپت تایپ ها رو استنتاج کنه!!

TypeScript می‌تواند بسیاری از تایپ‌ها را به‌صورت خودکار تشخیص دهد؛ بنابراین، نوشتن Type Annotation در تمام موقعیت‌ها ضروری نیست.

```
const username = "john"; // string
const age = 18; // number
```

تکرار تایپی که به‌وضوح قابل استنباط است، معمولا ارزش زیادی ندارد؛ اما Type Annotation در موقعیت‌هایی مانند پارامترهای تابع، آرایه‌های خالی و متغیرهایی که تایپ اولیه مشخصی ندارند، می‌تواند ضروری یا مفید باشد.

به‌جای حذف همه Type Annotationها، فقط زمانی از آن‌ها استفاده کنید که به وضوح، دقت یا کنترل بهتر تایپ‌ها کمک می‌کنند.

مقادیر مجاز را تا جای ممکن محدود نگه دارید!**

وقتی یک مقدار فقط می‌تواند یکی از چند گزینه مشخص باشد، به‌جای استفاده از تایپ‌های گسترده‌ای مثل `string`، از Literal Type یا Union Type استفاده کنید.

```
type Theme = "light" | "dark";
type Status = "loading" | "success" | "error";
```

این کار باعث می‌شود TypeScript مقادیر نامعتبر را تشخیص دهد و تایپ‌چکینگ دقیق‌تری داشته باشید.

کلا نه فقط لیترال تایپ ها، بلکه کلا همیشه همیشه محدود کار کنید! به قولی پروژتون سخت‌گیررررررر باشه!!!

از قوانین ESLint برای یکدست شدن سبک کد استفاده کنید

بعضی از کانفیگ‌ها و پکیج‌های ESLint، مانند `@typescript-eslint/consistent-type-definitions`، می‌توانند استفاده از `interface` یا `type` را برای تعریف ساختار آبجکت‌ها یکدست کنند و در موارد قابل اصلاح، تبدیل خودکار انجام دهند.

با این حال، این قانون تمام Type Aliasها را به Interface تبدیل نمی‌کند؛ چون تایپ‌هایی مانند Union و Tuple کاربردها و ساختار متفاوتی دارند.

هدف، استفاده از ابزار مناسب برای هر مسئله است؛ نه استفاده افراطی از یک روش. تایپ‌های دقیق، استنباط مناسب و رعایت یک سبک ثابت در پروژه، همگی به خوانایی و نگه‌داری بهتر کد کمک می‌کنند.

از Literal Typeها استفاده کنید

تا جای ممکن، تایپ‌ها را دقیق و محدود تعریف کنید. وقتی مقدارهای مجاز مشخص هستند، به‌جای تایپ‌های گسترده‌ای مثل `string` یا `number`، از Literal Type یا Union از Literalها استفاده کنید.

```
type Status = "loading" | "success" | "error";
type Theme = "light" | "dark";
type ButtonSize = "small" | "medium" | "large";
```

این کار باعث می‌شود TypeScript مقدارهای نامعتبر را تشخیص دهد، تایپ‌چکینگ دقیق‌تری داشته باشید و از خطاهای احتمالی جلوگیری کنید.

نکته: تایپ را فقط به اندازه نیاز محدود کنید؛ برای مقادیری که واقعا آزادانه تغییر می‌کنند، مانند نام کاربر یا متن ورودی، استفاده از `string` مناسب‌تر است.

برای مدیریت تایپ APIها از Specification استفاده کنید

برای مدیریت تایپ مسیرهای API، می‌توانید یک Specification تعریف کنید که هر مسیر را به تایپ پاسخ آن مرتبط کند. به این ترتیب، تغییرات مسیرها و تایپ پاسخ‌ها در یک محل مدیریت می‌شوند.

```
interface ClassbonAPI {
  "/users": User[];
  "/user/:id": User;
  "/courses": Course[];
  "/course/:id": Course;
}
```

با استفاده از `keyof` و Genericها می‌توان تابعی ساخت که مسیر معتبر را بپذیرد و تایپ پاسخ را بر اساس همان مسیر تعیین کند.

```
declare class ApiFetcher<API> {
  fetch<Path extends keyof API>(path: Path): Promise<API[Path]>;
}

const fetcher = new ApiFetcher<ClassbonAPI>();

const users = fetcher.fetch("/users"); // Promise<User[]>
const course = fetcher.fetch("/courses"); // Promise<Course[]>
```

نکته: این روش تایپ مسیرها و پاسخ‌ها را در زمان توسعه بررسی می‌کند؛ اما به‌تنهایی صحت پاسخ واقعی سرور را در زمان اجرا تضمین نمی‌کند.

برای تولید خودکار تایپ API از Orval استفاده کنید!!!

با توجه به نکته ی قبلی، وقتی پروژه تعداد زیادی Endpoint دارد یا مشخصات API مرتب تغییر می‌کند، نگهداری دستی تایپ‌ها و توابع درخواست می‌تواند زمان‌بر و مستعد خطا باشد.

درباب همین موضوع ابزار `Orval` می‌تواند بر اساس یک فایل OpenAPI، کد و تایپ‌های مربوط به API را تولید کند. همچنین امکان تولید توابع درخواست و استفاده از قابلیت‌های TanStack Query را فراهم می‌کند.

به‌جای اینکه برای هر Endpoint تایپ‌ها و توابع را دستی بنویسید، می‌توانید مشخصات API را به‌روز نگه دارید و کد مورد نیاز را دوباره تولید کنید.

برای APIهای بزرگ یا پروژه‌هایی که قرارداد API آن‌ها مرتب تغییر می‌کند، تولید خودکار کد را بررسی کنید؛ البته خروجی تولیدشده باید با ساختار و نیازهای پروژه هماهنگ باشد.

برای تعریف مدل‌ها از Interface و برای تایپ‌های ترکیبی از Type Alias استفاده کنید

برای تعریف ساختار آبجکت‌ها، `interface` انتخاب مناسبی است. برای ترکیب تایپ‌ها، تعریف Unionها و ساخت تایپ‌های پیچیده، `type` انعطاف بیشتری دارد.

```
interface Course {
  id: number;
  title: string;
}

interface Quiz {
  id: number;
  questions: string[];
}

type Content = Course | Quiz;

type ContentWithMetadata = Content & {
  createdBy: string;
  createdAt: Date;
};
```

در این مثال، `Content` یک Union Type است و `ContentWithMetadata` با استفاده از Intersection Type، اطلاعات مشترک را به هر دو حالت اضافه می‌کند.

برای ساختارهای ساده آبجکت می‌توانید از `interface` و برای Union، Intersection و تایپ‌های ترکیبی از `type` استفاده کنید. این ترجیح یک راهنمای عملی است، نه یک الزام مطلق.

برای تعریف تایپ تابع، سادگی و کاربرد را در نظر بگیرید!*

برای تعریف تایپ یک تابع، هم `type` و هم `interface` قابل استفاده‌اند. برای امضای ساده و قابل استفاده مجدد، Type Alias معمولا انتخاب سرراستی است.

```
type StringTransform = (input: string) => string;

const toLowerCase: StringTransform = (str) => str.toLowerCase();
```

همچنین می‌توان امضای تابع را با Interface تعریف کرد:

```
interface StringTransform {
  (input: string): string;
}
```

هر دو روش معتبرند. انتخاب بین آن‌ها باید با توجه به خوانایی، نیازهای تایپی و سبک پروژه انجام شود.

قانون: برای تایپ‌های تابع، روشی را انتخاب کنید که امضا را واضح‌تر کند و با قراردادهای پروژه سازگار باشد.

از Currying برای تقسیم فراخوانی تابع استفاده کنید!

Currying روشی است که در آن تابعی با چند ورودی، به زنجیره‌ای از توابع تبدیل می‌شود که هر کدام یک ورودی دریافت می‌کنند.

```
const calculateArea =
  (length: number) ⇒
  (width: number): number ⇒
    length * width;

const calculateWidth = calculateArea(5);
const area = calculateWidth(10); // 50
```

در این مثال، فراخوانی اول `length` را دریافت می‌کند و تابعی برمی‌گرداند که `width` را می‌گیرد.

نکته: Currying همیشه بهتر از تابع چندپارامتری نیست. زمانی از آن استفاده کنید که جدا کردن مراحل فراخوانی یا نگه داشتن یک ورودی برای استفاده مجدد، واقعا به طراحی کد کمک می‌کند.

نام‌گذاری را با اصطلاحات دامنه برنامه هماهنگ کنید!

نام متغیرها، پراپرتی‌ها، توابع و تایپ‌ها باید با مفهوم واقعی آن‌ها در پروژه هماهنگ باشد. از نام‌های مبهم یا اصطلاحاتی که با مدل کسب‌وکار سازگار نیستند، دوری کنید.

برای مثال، اگر در سیستم آموزشی از مفهوم «مدرس» استفاده می‌کنید، نام `teacherId` ممکن است از `instructorId` برای تیم شما واضح‌تر باشد؛ البته اگر واژه رسمی دامنه پروژه `instructor` است، همان را به کار ببرید.

```
interface Course {
  id: number;
  teacherId: number;
}
```

نام‌گذاری را بر اساس واژگان واقعی دامنه پروژه انجام دهید و برای یک مفهوم واحد، در بخش‌های مختلف کد از نام‌های متفاوت استفاده نکنید.

کد را برای توسعه‌دهنده بعدی ساده و خوانا بنویسید!

Clean Code فقط به کوتاه بودن کد یا استفاده از قابلیت‌های پیشرفته زبان محدود نیست. کد خوب باید برای اعضای تیم قابل فهم و نگهداری باشد.

- از نام‌های روشن و متناسب با مفهوم استفاده کنید.
- از پیچیدگی غیرضروری و انتزاع‌های بی‌دلیل پرهیز کنید.
- برای هر قابلیت پیشرفته، دلیل مشخصی داشته باشید.
- قراردادهای نام‌گذاری و سبک کدنویسی پروژه را یکدست نگه دارید.

هدف این نیست که کد از نظر ظاهری پیچیده یا حرفه‌ای به نظر برسد؛ هدف این است که توسعه‌دهنده بعدی بتواند منطق آن را سریع و درست درک کند.

فکر می‌کنی بدنه `interface` حتما باید پروپرتی‌های یک آبجکت رو تعریف کنه؟!

در TypeScript، اینترفیس می‌تونه شکل یک تابع یا حتی یک آرایه رو هم توصیف کنه.

- **مثال اینترفیس برای آرایه**

```ts
interface StringArray {
  [index: number]: string;
  length: number;
}

const names: StringArray = ["Ali", "Sara", "Reza"];

console.log(names[0]); // "Ali"
console.log(names.length); // 3
```

یا حتی می‌تونی بجای ایندکس سیگنچر مستقیم پروپرتی هاشو با عدد تنظیم کنی مثل 0: 1: 2: و..!

- **فانکشن**

```ts
interface User {
  name: string;
  greet: (message: string) => void;
}

const user: User = {
  name: "Ali",
  greet: (message) => {
    console.log(message);
  },
};
```

اینجا `greet` یک پروپرتی از نوع تابع است؛ پس داخل `interface` به شکل یک پروپرتی تعریف شده.

اما این مثال:

```
interface StringTransform {
  (input: string): string;
}
```

Type Narrowing

تایپ‌اسکریپت با بررسی شرط‌ها می‌تونه نوع دقیق‌تر یک متغیر رو در بخش‌های مختلف کد تشخیص بده. به این فرایند **Type Narrowing** می‌گن.

```
function printValue(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  } else {
    console.log(value.toFixed(2));
  }
}
```

در این مثال، نوع `value` ابتدا `string | number` است؛ اما داخل شرط اول، تایپ‌اسکریپت می‌فهمه که نوع آن `string` است و در شاخه `else` نوع آن `number` خواهد بود.

روش‌های رایج Type Narrowing:

- `typeof`
- `instanceof`
- بررسی مقدار با `===` و `!==`
- بررسی وجود پروپرتی با `in`
- Discriminated Union
- Type Guardهای سفارشی

نکته: Type Narrowing نوع متغیر را در زمان بررسی نوع دقیق‌تر می‌کنه و لزوما مقدار آن را تغییر نمی‌ده.

تفاوت Interface و Type در TypeScript

تفاوت ها و شباهت های مختلفی دارن اما چون از قبل می‌دونم که خودتون بلدید من دیگه اینجا خیلی روشون مانور ندادم!

هر دو `interface` و `type` برای تعریف تایپ‌ها استفاده می‌شن، اما قابلیت‌ها و محدودیت‌های متفاوتی دارن.

1. چرا Interface نمی‌تونه Union Type رو Extend کنه؟

```
type Admin = {
  role: "admin";
};

type User = {
  role: "user";
};

type Account = Admin | User;
```

حالا اگر بنویسیم:

```
interface ExtendedAccount extends Account {
  id: string;
}
```

TypeScript خطا می‌ده؛ چون `Account` یک Union Type است و می‌تونه یکی از چند ساختار متفاوت باشه. `interface extends` برای گسترش یک ساختار مشخص طراحی شده، نه ارث‌بری از چند حالت جایگزین.

اما با `type` می‌تونیم از Intersection استفاده کنیم:

```ts
type ExtendedAccount = Account & {
  id: string;
};
```

در این حالت، هر عضو `Account` با ساختار دارای `id` ترکیب می‌شه.

2. تفاوت‌های اصلی

Interface:

- برای تعریف شکل آبجکت‌ها و قراردادهای ساختاری مناسبه.
- می‌تونه با `extends` از اینترفیس‌های دیگر گسترش پیدا کنه.
- از قابلیت Declaration Merging پشتیبانی می‌کنه؛ یعنی تعریف‌های هم‌نامش می‌تونن ادغام بشن.
- نمی‌تونه مستقیما یک Union Type یا Tuple Type تعریف کنه.

Type:

- می‌تونه شکل آبجکت‌ها رو تعریف کنه.
- برای تعریف Tuple، Intersection، Union و تایپ‌های ترکیبی کاربرد داره.
- می‌تونه با `&` چند تایپ رو ترکیب کنه.
- از Declaration Merging پشتیبانی نمی‌کنه.

2. چرا تایپ اسکریپت این تفاوت رو داره؟!

از نظر طراحی زبان، `interface` برای تعریف قراردادهای ساختاری و قابل گسترش مناسبه؛ در مقابل، `type` امکان نام‌گذاری و ترکیب انواع مختلف تایپ‌ها رو فراهم می‌کنه.

بااینحال نباید ادعا کنیم که دلیل تاریخی‌ودقیق طراحان TS برای تک‌تک این محدودیت‌ها مشخصه؛ توضیح بالا تفاوت مفهومی و کاربردی این دو ابزار رو بیان می‌کنه. 🤦

برای تعریف شکل یک آبجکت، معمولا می‌تونیم از `interface` شروع کنیم. وقتی به Union، Tuple یا ترکیب‌های پیچیده‌تر نیاز داریم، `type` انتخاب مناسبیه.

هیچ‌کدوم به‌صورت مطلق بهتر نیستن؛ انتخاب به ساختار تایپ و نیاز پروژه بستگی داره.

طراحی تایپ (Type Design)

در این فصل به اصول و بهترین روش‌های طراحی Type در TypeScript پرداخته می‌شه.

مباحث کلیدی شامل استراتژی های طراحی بهینه تایپ‌ها، اجتناب از استفاده مستقیم از null و undefined، و اولویت دادن به Union Types از Interfaceها بررسی

می‌شن.

همچنین تأکید می‌شه که چگونه می‌توان تایپ های string را محدودتر کرد، از پارامترهای تکراری با تایپ یکسان اجتناب نمود، و در استفاده از APIها همیشه از تایپ های مشخص استفاده کرد.

در نهایت، مزایای استفاده از unknown به جای any و اهمیت محدود کردن دامنه تایپ‌ها بررسی می شن.

در تعریف تایپ ها هیچ وقت از null و undefined به شکل مستقیم استفاده نکنید!

وقتی یک مقدار باید حتما وجود داشته باشه، نباید تایپ رو بی‌دلیل با `null` یا `undefined` گسترش بدیم.

```
interface User {
  id: number;
  name: string;
}

function getUserInfo(user: User) {
  console.log(user.name);
}

getUserInfo(null); // خطای تایپی در حالت strictNullChecks
```

اگر `null` رو به‌عنوان یک ورودی معتبر اعلام نکرده باشیم، TypeScript اجازه ارسال اون رو نمی‌ده.

اما اگر نبودن مقدار بخشی از منطق برنامه باشه، باید اون رو صریح مدل کنیم:

```
function getUserInfo(user: User | null) {
  if (user === null) {
    return;
  }

  console.log(user.name);
}
```

نکته: استفاده از `null` و `undefined` ذاتا اشتباه نیست. هدف اینه که فقط زمانی از اون‌ها استفاده کنیم که نبودن مقدار واقعا در منطق برنامه معنی داشته باشه. فعال بودن `strictNullChecks` هم برای تشخیص این موارد مهمه.

استخراج تایپ و حفظ Literalها

به‌جای تعریف دستی یک تایپ که از قبل در یک متغیر وجود داره، می‌تونیم از `typeof` در جایگاه تایپ استفاده کنیم.

```
const course = {
  title: "JavaScript",
  status: "published",
};

type Course = typeof course;
```

در این مثال، تایپ `Course` از ساختار متغیر `course` استخراج می‌شه. با این حال، TypeScript معمولا مقادیر رشته‌ای آبجکت رو به `string` تعمیم می‌ده.

استفاده از as const برای حفظ Literalها

```
const course = {
  title: "JavaScript",
  status: "published",
} as const;
```

حالا تایپ `status` به‌جای `string` برابر با `"published"` است و پروپرتی‌ها در تایپ به‌صورت `readonly` در نظر گرفته می‌شن.

برای استخراج مقادیر مجاز از یک آرایه:

```
const statuses = ["published", "draft", "archived"] as const;

type CourseStatus = (typeof statuses)[number];
```

تایپ `CourseStatus` برابر با این Union خواهد بود:

```
type CourseStatus = "published" | "draft" | "archived";
```

`as const` آرایه رو در سطح تایپ به یک Tuple فقط‌خواندنی تبدیل می‌کنه؛ اما در زمان اجرا، آرایه رو فریز نمی‌کنه.

استفاده از readonly برای جلوگیری از تغییر تایپی

```
const importanceLevel: readonly [1, 2, 3] = [1, 2, 3];

// خطا
importanceLevel[0] = 10;
```

در این مثال، هم موقعیت عناصر و هم مقادیر مجاز اون‌ها مشخص هستن. همچنین می‌تونیم با `as const` همین ساختار رو کوتاه‌تر بنویسیم:

```
const importanceLevel = [1, 2, 3] as const;
```

استفاده از satisfies – بررسی سازگاری تایپ بدون جایگزین کردن نوع استنباط‌شده

عملگر `satisfies` بررسی می‌کنه که یک مقدار با تایپ موردنظر سازگاره، اما نوع نهایی متغیر رو مستقیما با اون تایپ جایگزین نمی‌کنه.

```
type Point = [number, number];

const locations = {
  tehran: [35.7, 51.4],
  karaj: [35.8, 50.9],
} satisfies Record<string, Point>;
```

در این مثال، TypeScript بررسی می‌کنه که مقدار هر کلید با `Point` سازگار باشه. همچنین اطلاعات دقیق‌تر نوع مقدارها تا حد ممکن حفظ می‌شن.

در مقابل:

```
const locations: Record<string, Point> = {
  tehran: [35.7, 51.4],
};
```

در اینجا نوع متغیر مستقیما `Record<string, Point>` تعریف شده؛ بنابراین می‌تونیم کلیدهای دیگری هم اضافه کنیم:

```
locations.karaj = [35.8, 50.9];
```

نکته: `satisfies` به معنی ثابت یا فقط‌خواندنی کردن مقدار نیست. اگر به محدودیت تغییر نیاز داریم، باید از ابزارهایی مثل `readonly` یا `as const` استفاده کنیم.

ترکیب تایپ‌ها و یک خطای مهم

هنگام استفاده از Intersection به تداخل تایپ‌ها توجه کنید!

فرض کنید:

```
interface User {
  id: number;
  name: string;
}
```

این ترکیب مشکل داره:

```
type AdminUser = User & {
  id: string;
  isSuperAdmin: boolean;
};
```

چون `id` در `User` از نوع `number` است، اما در بخش دوم از نوع `string` تعریف شده. یک مقدار نمی‌تونه هم‌زمان هر دو نوع ناسازگار رو داشته باشه؛ بنابراین نوع `id` به `never` می‌رسه.

راه‌حل مناسب، حذف تایپ قبلی `id` و تعریف مجدد اونه:

```
type AdminUser = Omit<User, "id"> & {
  id: string;
  isSuperAdmin: boolean;
};
```

حالا `AdminUser` دارای `name`، یک `id` رشته‌ای و `isSuperAdmin` از نوع `boolean` است.

در TypeScript می‌تونی برای اعضای Tuple اسم هم مشخص کنی!***

سینتکس آن به این شکل است:

```
type nuns = [num1: 10, num2: 20];
```

در این مثال:

- `num1` و `num2` اسم اعضای Tuple هستند و فقط برای خوانایی بهتر و راهنمایی در ادیتور استفاده می‌شوند.
- `10` و `20` نوع اعضا هستند و چون Literal Type هستند، فقط همین مقادیر پذیرفته می‌شوند.

```
const t1: nums = [10, 20]; // درست
const t2: nums = [10, 30, 7]; // خطا
```

اگر بخواهی هر عددی پذیرفته شود، به‌جای `10` و `20` از `number` استفاده کن:

```
type nums = [num1: number, num2: number];
```

یک مثال کاربردی از استفاده‌اش می‌تونه یه کاستوم هوک ری اکت باشه که یه استیت و setter ریتورن می‌کنه! اینجا می‌تونید به ارایه برگشتی‌تون تگ بدید.

نکته: اسم‌گذاری اعضای Tuple روی مقدارها یا رفتار آن‌ها تاثیری ندارد و فقط خوانایی کد را بهتر می‌کند.

هیچ‌وقت پارامتر هایی با یک تایپ مشخص را پشت سرهم تعریف نکنید!

هنگام طراحی توابع، بهتر است از تعریف چند پارامتر متوالی با تایپ یکسان خودداری کنید؛ به‌خصوص زمانی که این پارامترها معنای متفاوتی دارند. در چنین شرایطی، احتمال جابه‌جایی آرگومان‌ها هنگام فراخوانی تابع افزایش می‌یابد، زیرا TypeScript نمی‌تواند مقادیر هم‌نوع را از نظر معنایی از یکدیگر تشخیص دهد.

برای مثال:

```
function createUser(
  firstName: string,
  lastName: string,
) {
  // ...
}

createUser("Mohammad", "Ahmadi");
```

در این مثال، اگر آرگومان‌ها به اشتباه جابه‌جا شوند، TypeScript خطایی گزارش نمی‌کند؛ زیرا هر دو مقدار از نوع `string` هستند.

یکی از راهکارهای مناسب، استفاده از یک شیء با پراپرتی‌های نام‌گذاری‌شده است:

```
function createUser({
  firstName,
  lastName,
}: {
  firstName: string;
  lastName: string;
}) {
  // ...
}

createUser({ firstName: "Mohammad", lastName: "Ahmadi" });
```

با این روش، هر مقدار با نام مشخصی ارسال می‌شود و احتمال اشتباه در ترتیب آرگومان‌ها کاهش می‌یابد.

البته این قاعده به معنای ممنوع بودن تعریف پارامترهای هم‌نوع نیست. اگر پارامترها ساده باشند و ترتیب آن‌ها کاملا مشخص باشد، تعریف آن‌ها به‌صورت جداگانه کاملا قابل قبول است.

بنابراین یه قاعده کلی براش می‌تونیم بچینیم: زمانی که چند پارامتر هم‌نوع معنای متفاوتی دارند و جابه‌جایی آن‌ها می‌تواند باعث بروز خطا شود، استفاده از یک شیء با پراپرتی‌های مشخص را در نظر بگیرید. هدف، افزایش خوانایی و کاهش خطاهای احتمالی است، نه حذف تمام پارامترهای هم‌نوع.

در زمان استفاده از APIها، برای یک API همیشه از یک تایپ مشخص استفاده کنید! هیچ وقت نباید چند تایپ تعریف کنید.

استفاده از یک تایپ مشترک برای هر قرارداد API*

هنگام کار با APIها، بهتر است برای هر ساختار مشخص از داده‌ها، یک تایپ مشترک تعریف کنید و در بخش‌های مختلف برنامه از همان تایپ استفاده کنید. تعریف چند تایپ با ساختار یکسان برای یک نوع داده، باعث تکرار کد، دشوارتر شدن نگهداری و افزایش احتمال ناهماهنگی هنگام تغییر ساختار داده‌ها می‌شود.

برای مثال، فرض کنید چند API اطلاعات کاربران را با ساختار یکسانی برمی‌گردانند. به‌جای تعریف تایپ جداگانه برای پاسخ هر API، می‌توانید یک تایپ مشترک تعریف کنید:

```
type User = {
  id: number;
  name: string;
  email: string;
};

function getUser(): Promise<User> {
  // ...
}

function getUsers(): Promise<User[]> {
  // ...
}
```

در این مثال، هر دو تابع از تایپ `User` استفاده می‌کنند؛ بنابراین اگر ساختار اطلاعات کاربر تغییر کند، کافی است تایپ اصلی را به‌روزرسانی کنید.

البته استفاده از یک تایپ مشترک همیشه الزامی نیست. اگر ساختار یا معنای داده‌های ورودی و خروجی APIها متفاوت باشد، بهتر است تایپ‌های جداگانه تعریف کنید. همچنین ممکن است یک API برای درخواست، پاسخ موفق و پاسخ خطا به تایپ‌های متفاوتی نیاز داشته باشد.

از تعریف تایپ‌های تکراری برای ساختارهای یکسان خودداری کنید و تایپ‌های مشترک را مجددا استفاده کنید؛ اما برای ساختارهای متفاوت، تایپ‌های مستقل و متناسب با هر قرارداد تعریف کنید.

مفهوم Evolving Type

Evolving Type به معنی «تغییر تدریجی نوع» است. در TypeScript، گاهی هنگام تعریف یک متغیر بدون مقدار اولیه یا با یک آرایه خالی، نوع دقیق آن در ابتدا مشخص نیست. در چنین شرایطی، TypeScript می‌تواند با توجه به مقادیری که بعدا به متغیر اختصاص داده می‌شوند، نوع آن را به‌تدریج مشخص کند.

برای مثال:

```
let value;

value = 10;       // number
value = "Ali";    // string
```

در این مثال، متغیر `value` در ابتدا نوع مشخصی ندارد و TypeScript آن را به‌صورت ویژه مدیریت می‌کند. البته این رفتار به معنای آن نیست که تمام متغیرها همیشه با تغییر مقدار، نوع خود را تغییر می‌دهند.

مثال با آرایه خالی:

```
function makeSquares() {
  const nums = [];

  [1, 2, 3].forEach((num) => {
    nums.push(num * num);
  });

  return nums;
}
```

در این مثال، TypeScript می‌تواند با مشاهده مقادیری که به آرایه `nums` اضافه می‌شوند، نوع آن را به‌تدریج مشخص کند. این رفتار یکی از نمونه‌های Evolving Type است.

نکته مهم:

Evolving Type با تغییر نوع یک متغیر در زمان اجرا تفاوت دارد. این مفهوم مربوط به نحوه استنباط نوع توسط TypeScript هنگام تحلیل کد است و فقط در شرایط مشخصی اتفاق می‌افتد.

در برخی موقعیت‌ها، TypeScript نوع متغیر را با توجه به نحوه استفاده از آن در طول کد به‌تدریج مشخص می‌کند؛ بنابراین همیشه لازم نیست نوع چنین متغیرهایی را از ابتدا به‌صورت صریح تعریف کنید.

نکته مهم درباره Boolean Literal Types

در TypeScript، اگر مقدار `true` یا `false` رو با `as const` یا در یک نوع مشخص استفاده کنیم، می‌تونیم به‌جای نوع کلی `boolean`، نوع دقیق همون مقدار رو داشته باشیم:

```
interface Config {
  enabled: true;
}
```

در این مثال، `enabled` از نوع `boolean` نیست؛ بلکه از نوع دقیق `true` است. بنابراین مقدار `false` برای اون مجاز نیست.

اگر هر دو مقدار `true` و `false` مجاز باشن، باید از `boolean` استفاده کنیم:

```
interface Config {
  enabled: boolean;
}
```

گاهی لازم است یک متغیر را هم با `const` تعریف کنید و هم از `as const` استفاده کنید؛ به‌خصوص هنگام کار با آبجکت‌ها

```
const config = {
  status: "active",
  retries: 3,
} as const;
```

در این مثال، `const` مانع از انتساب مجدد متغیر `config` می‌شود؛ اما `as const` باعث می‌شود ویژگی‌های آبجکت به‌صورت `readonly` و با نوع‌های دقیق‌تر استنباط شوند.

برای مثال، نوع `config.status` به‌جای `string`، برابر با `"active"` خواهد بود و نمی‌توان مقدار ویژگی‌های آبجکت را تغییر داد.

نکته: `as const` در زمان اجرا آبجکت را فریز نمی‌کند؛ این محدودیت‌ها مربوط به بررسی نوع در TypeScript هستند.

درواقع، `const` جلوی **Reassignment** متغیر رو می‌گیره، اما مانع **Mutation** آبجکت نمی‌شه!* در مقابل، `as const` باعث می‌شه ویژگی‌های آبجکت در سطح TypeScript به‌صورت `readonly` تعریف بشن و تغییر یا **Mutation**اون‌ها با خطای TypeScript مواجه بشه.

تعریف امضای تابع با Type و Interface

با `type` می‌توان یک نوع تابع تعریف کرد و سپس آن را به متغیر اختصاص داد:

```
type StringTransform = (input: string) => string;

const toLowerCase: StringTransform = (str) => str.toLowerCase();
```

همچنین می‌توان امضای تابع را مستقیما با `interface` تعریف کرد:

```
interface StringTransform {
  (input: string): string;
}
```

این دو روش برای تعریف نوع تابع معتبر هستند؛ اما `type` برای ترکیب انواع مختلف، مانند Conditional Type، Tuple، Intersection، Union و Mapped Type نیز کاربرد دارد.

نکته‌ای درباره Index Signature

در TypeScript، لازم نیست `Index Signature` آخر تعریف آبجکت قرار بگیرد. مسئله اصلی ترتیب نیست، بلکه سازگاری نوع مقادیر است.

```
type Scores = {
  [index: number]: string;
  name: string;
};
```

این کد معتبر است؛ چون `name` یک ویژگی معمولی است و `Index Signature` عددی فقط نوع ویژگی‌هایی را مشخص می‌کند که از طریق اندیس عددی قابل دسترسی هستند.

بنابراین، وجود `age: number` در کنار این `Index Signature` نیز به‌خودی‌خود خطا ایجاد نمی‌کند؛ چون `Index Signature` عددی، تمام ویژگی‌های نام‌دار رشته‌ای را محدود نمی‌کند.

اما اگه name از تایپ نامبر بود خطا می‌گرفتیم! زیرا قبلش گفتیمن تمام پرااپرتی ها باید از تایپ استرینگ باشند!

پرهیز از استفاده غیرضروری از Null و Undefined

هنگام تعریف نوع‌ها، بهتر است فقط زمانی `null` یا `undefined` را مجاز کنید که واقعا بخشی از قرارداد داده باشند.

```
type User = {
  id: number;
  name: string;
};
```

اگر `name` همیشه باید یک رشته باشد، نیازی نیست نوع آن را به‌صورت `string | null` تعریف کنید.

اما اگر ممکن است نام کاربر وجود نداشته باشد، می‌توانید این حالت را صریحا مشخص کنید، بهتر از روش های دیگه‌ای استفاده کنید!

چون مدل داده‌ها یه ساختاره! یا وجود داره یا نداره یه روش کاملا اشتباهی هستش! یعنی چی یوزر یا وجود داره یا نداره؟!

نکته این است که استفاده از `null` یا `undefined` به‌خودی‌خود اشتباه نیست؛ هدف این است که نوع‌ها با وضعیت‌های واقعی داده مطابقت داشته باشند و حالت‌های غیرضروری به قرارداد اضافه نشوند.

Evolving Array Types

گاهی TypeScript می‌تواند نوع آرایه‌ای را که ابتدا خالی تعریف شده، بر اساس مقادیری که بعدا به آن اضافه می‌شوند، استنباط کند. به این رفتار **Evolving Array** گفته می‌شود.

```
function makeSquares() {
  const nums = [];

  [1, 2, 3].forEach((i) => {
    nums.push(i * i);
  });

  return nums;
}
```

در شرایط مناسب، TypeScript می‌تواند نوع عناصر `nums` را از مقادیر اضافه‌شده استنباط کند. با این حال، رفتار آن به زمینه استفاده و نحوه بررسی کد بستگی دارد؛ بنابراین نباید فرض کرد که هر آرایه خالی در تمام شرایط به همین شکل تایپ می‌شود.

****

نکته‌ای که در مثال بالا وجود داره و خیلی مهمه، اینه که با حلقه فورایچ ts نمی‌تونه تایپ رو تنظیم کنه! حالا چرا؟ چون فورایچ یه کال‌بک فانکشن بر می‌گردونه، و خوب این هم برای تایپ اسکریپن هزینه داره که بخواد بررس یکنه تمام کال بک فانکمشن های پروژه چه زمانی اجرا میشن؟

پس راه حل چیه؟ از حلقه for of استفاده کنید!!!

****

یه چالش باحال! ;)

```
// Code 1
createAdmin();
createUser();

// Code 2
createAccount(type);
```

کدوم کد رو ترجیح می‌دین؟

یکم راجبش فکر کنید بعد برید سراغ ادامه!!!!.

.

.

.

.

.

.

.

.

طبق آماری که قبلا از دولوپرهای مختلف گرفتم، اکثر دولوپرهای کم‌تجربه‌تر گزینه 2 رو انتخاب می‌کنن!

در رابطه با این Poll باید به یه مفهوم به اسم **Best Practice Trap** یا «تله Best Practiceها» اشاره کنم.

خیلی وقت‌ها وقتی با یه Best Practice آشنا می‌شیم، اون رو به‌عنوان یه قانون مطلق در نظر می‌گیریم؛ انگار همیشه درسته و باید همه‌جا ازش استفاده کنیم.

توی مثال بالا، احتمالا دلیل انتخاب کد 2 برای یه دولوپر کم‌تجربه، آشنایی با اصل DRY باشه. با خودش می‌گه چرا دو تا تابع جدا داشته باشیم وقتی می‌تونیم همه‌چیز رو با یه تابع مدیریت کنیم؟

اما مسئله اینجاست که ممکنه تابع `createAccount` به‌مرور پر از شرط‌های `if` بشه. از طرفی، نیازهای کاربر و ادمین ممکنه در آینده از هم جدا بشن و هرکدوم منطق مخصوص خودشون رو پیدا کنن. در نتیجه، تابعی که قرار بود کد رو ساده‌تر کنه، کم‌کم به یه تابع پیچیده تبدیل می‌شه که تغییر دادنش هم سخت‌تره.

گاهی اوقات **کمی تکرار کد بهتر از Abstraction زودهنگامه.**

پس سعی کنید توی تله Best Practiceها نیفتید. Best Practiceها فکت‌های مطلق نیستن؛ بیشترشون در یک Context مشخص معنا پیدا می‌کنن و چیزی که توی یک پروژه جواب می‌ده، لزوما بهترین انتخاب برای پروژه دیگه‌ای نیست.

مهم اینه که بدونیم چرا از یک اصل استفاده می‌کنیم، نه اینکه صرفا چون اسمش Best Practiceـه، کورکورانه ازش پیروی کنیم. 🤦

Index Signature و تفاوت نوع پراپرتی‌ها

در TypeScript، وقتی از `Index Signature` با کلید مثلا `string` استفاده می‌کنید، نوع مقدار تمام پراپرتی‌های نام‌دار آبجکت باید با نوع مشخص‌شده در آن سازگار باشد.

```
type Scores = {
  name: number;
  [index: string]: string;
};
```

این کد خطا می‌دهد؛ چون `name` از نوع `number` است، اما `Index Signature` مشخص کرده که مقادیر پراپرتی‌های رشته‌ای باید از نوع `string` باشند.

نکته: ترتیب تعریف پراپرتی‌ها و `Index Signature` تفاوتی ایجاد نمی‌کند؛ بنابراین جابه‌جایی آن‌ها خطا را برطرف نمی‌کند.

اگر می‌خواهید پراپرتی‌ها بتوانند مقادیر متفاوتی داشته باشند، می‌توانید از Union Type استفاده کنید:

```
type Scores = {
  name: number;
  [index: string]: string | number;
};
```

حالا پراپرتی‌ها می‌توانند مقدار `string` یا `number` داشته باشند و `name` نیز با این ساختار سازگار است.

اینم یه نکته ریزی بود که گفتم بگم، البته احتمالا این رو از قبل خودتون می‌دونستید!

استخراج Union از اعضای آرایه در TypeScript

```
const statuses = ["published", "draft", "archived"] as const;

type CourseStatus = (typeof statuses)[number];
```

با استفاده از `as const`، مقادیر آرایه به‌صورت Literal Type حفظ می‌شن. سپس `[number](typeof statuses)` نوع تمام اعضای آرایه رو استخراج می‌کنه و به شکل یک Union Type درمیاره:

```
type CourseStatus = "published" | "draft" | "archived";
```

به این ترتیب، `CourseStatus` فقط یکی از این سه مقدار رو می‌پذیره.

تا حد امکان از Typeهای محدودتر استفاده کنید

نوع‌های عمومی مثل `string` در بسیاری از سناریوها بیش از حد گسترده‌اند. بهتر است تا جایی که امکان دارد، نوع‌ها را به مقادیر معتبر محدود کنید تا TypeScript بتواند خطاهای بیشتری را تشخیص دهد.

```
type CourseStatus = "published" | "draft" | "archived";
```

به‌جای استفاده از `string` برای وضعیت دوره، این نوع فقط مقادیر معتبر را می‌پذیرد و از ورود مقادیر نامعتبر جلوگیری می‌کند.

هر Type باید یک State معتبر را نشان دهد

هنگام طراحی Typeها، سعی کنید هر Type نشان‌دهنده یک وضعیت معتبر و مشخص باشد. Typeهایی که چند وضعیت متفاوت را به‌صورت مبهم ترکیب می‌کنند، ممکن است باعث خطا و پیچیدگی شوند.

برای مثال، اگر نتیجه یک عملیات یا موفق است یا ناموفق، بهتر است این دو حالت را به‌صورت یک Union مشخص تعریف کنید:

```
type Result =
  | { status: "success"; data: string }
  | { status: "error"; message: string };
```

در این ساختار، هر حالت ویژگی‌های متناسب با خودش را دارد و TypeScript می‌تواند بر اساس `status` نوع دقیق هر حالت را تشخیص دهد.

به‌طور کلی، Typeهایی را ترجیح دهید که فقط وضعیت‌های معتبر را نمایش می‌دهند؛ حتی اگر تعریف آن‌ها کمی طولانی‌تر باشد.

برای هر API، قرارداد تایپی و نوعی مشخصی داشته باشید

هنگام کار با APIها، بهتر است قرارداد داده‌های دریافتی را به‌صورت مشخص و یکپارچه تعریف کنید و از ایجاد Typeهای تکراری و غیرضروری برای یک ساختار یکسان خودداری کنید.

برای مثال، اگر پاسخ API شامل ویژگی `course_title` است، بهتر است تایپ پاسخ را مطابق قرارداد واقعی API تعریف کنید، نه اینکه بدون نیاز یک Type دیگر با نام `CourseDto` و ساختار تکراری بسازید.

البته اگر ساختار داده بین لایه‌های مختلف برنامه واقعا متفاوت است، تعریف Type جداگانه برای مدل داخلی و تبدیل داده‌ها می‌تواند منطقی باشد.

دامنه استفاده از Any را محدود کنید

حتی اگر در شرایط خاصی مجبور به استفاده از `any` شدید، دامنه استفاده از آن را تا حد امکان محدود نگه دارید. `any` بخش مهمی از بررسی نوع را برای مقدار مربوطه کنار می‌گذارد و می‌تواند باعث عبور خطاهای تایپی از بررسی کامپایل شود.

هرچه `any` در بخش‌های بیشتری از برنامه پخش شود، مزایای TypeScript کمتر خواهد شد.

Uncanny Valley of Type Safety🤔

اصطلاح **Uncanny Valley of Type Safety** را می‌توان برای توصیف وضعیتی به کار برد که Typeها در ظاهر بسیار دقیق و مطمئن به نظر می‌رسند، اما هنوز تمام حالت‌های واقعی داده را پوشش نمی‌دهند.

در چنین شرایطی، ممکن است به دلیل اعتماد بیش از حد به تعریف‌های تایپی، تصور کنیم برنامه کاملا ایمن است؛ درحالی‌که بعضی حالت‌های واقعی یا غیرمنتظره همچنان مدیریت نشده‌اند.

هدف، ساختن Typeهایی نیست که فقط پیچیده و دقیق به نظر برسند؛ بلکه باید تا حد امکان با داده‌ها و وضعیت‌های واقعی برنامه مطابقت داشته باشند.

*** از تعریف بیش از حد دقیق Typeها خودداری کنید! ***

با توجه به تعریف صفحه‌ی قبلی:

لزومی ندارد همیشه تمام جزئیات یک مدل را با بیشترین میزان دقت ممکن تعریف کنید. اگر نتوانستید ساختار داده‌ای را به‌طور کامل و دقیق مدل کنید، اشکالی ندارد که در بعضی بخش‌ها از Type عمومی‌تری استفاده کنید.

مهم این است که Typeها به‌اندازه کافی دقیق باشند تا خطاهای مهم را تشخیص دهند، اما آن‌قدر محدود و پیچیده نباشند که با تغییرات عادی داده‌ها یا شرایط واقعی Production ناسازگار شوند.

به بیان دیگر، بین دقت Typeها، پیچیدگی نگهداری و نیازهای واقعی برنامه تعادل برقرار کنید.

به‌جای Union کردن Interfaceها، از Union Type استفاده کنید!*

اگر چند ساختار آبجکتی دارید که هرکدام یک حالت معتبر را نشان می‌دهند، می‌توانید هر ساختار را با `interface` تعریف کنید و سپس با `type` آن‌ها را به شکل Union ترکیب کنید.

```
interface Success {
  status: "success";
  data: string;
}

interface Failure {
  status: "error";
  message: string;
}

type Result = Success | Failure;
```

در این مثال، هر `interface` یک ساختار مشخص را تعریف می‌کند و `Result` مشخص می‌کند که نتیجه می‌تواند یکی از این دو حالت باشد.

این روش باعث می‌شود هر حالت جداگانه قابل توسعه و نگهداری باشد و در عین حال، **مجموعه** حالت‌های ممکن به‌صورت یک Union Type مشخص شود.

البته در یسری مواقع بهتره که به‌جای تعریف اینترفیس‌هایی از تایپ union از union تایپ‌هایی از اینترفیس‌ها استفاده کنید!*

نوع داده‌های خارجی را بررسی کنید!

داده‌هایی که از منابع خارجی مثل `localStorage` دریافت می‌کنید، لزوما ساختار معتبری ندارند. برای مثال، `JSON.parse` ممکن است داده‌ای با ساختار غیرمنتظره برگرداند؛ بنابراین بهتر است قبل از استفاده از ویژگی‌های آن، اعتبار داده را بررسی کنید.

```
function getUserInfo(): UserInfo | null {
  const userInfoString = localStorage.getItem("userInfo");

  if (!userInfoString) {
    return null;
  }

  try {
    const userInfo: unknown = JSON.parse(userInfoString);

    if (isUserInfo(userInfo)) {
      return userInfo;
    }
  } catch {
    console.log("Invalid JSON in localStorage");
  }

  return null;
}
```

در این مثال، نوع `unknown` باعث می‌شود قبل از استفاده از داده، نوع آن بررسی شود. همچنین تابع `isUserInfo` باید ساختار داده را به‌درستی اعتبارسنجی کند.

Bottom Type & Top Type

- **Bottom Type:** تایپی که هیچ مقداری نمی‌تونه بهش نسبت داده بشه؛ یعنی `never`.
- **Top Type:** تایپی که همه مقادیر می‌تونن بهش نسبت داده بشن؛ مثل `unknown` و `any`.

تایپ `never` برای مقادیری استفاده می‌شود که امکان وجود ندارند یا هیچ مقداری نمی‌توان به آن‌ها اختصاص داد.

```
let x: never;

x = 10; // Error
```

`never` پایین‌ترین نوع در سیستم نوع TypeScript است (که به اصطلاح میگن یه **bottom type** هستش) و در بررسی حالت‌های غیرممکن، توابعی که هرگز برنمی‌گردند و بررسی کامل بودن حالت‌های یک Union کاربرد دارد.

از Type Guard استفاده کنید!

هنگام دریافت داده‌هایی با نوع `unknown`، می‌توانید از Type Guard برای بررسی ساختار داده استفاده کنید.

```
function isUserInfo(value: unknown): value is UserInfo {
  return (
    typeof value === "object" &&
    value !== null &&
    "permissions" in value
  );
}
```

عبارت `value is UserInfo` یک Type Predicate است که به TypeScript می‌گوید در صورت موفقیت بررسی، مقدار را از نوع `UserInfo` در نظر بگیرد.

البته برای اعتبارسنجی کامل، باید تمام ویژگی‌های ضروری `UserInfo` بررسی شوند؛ وجود یک ویژگی به‌تنهایی تضمین نمی‌کند که کل ساختار معتبر است.

****

از Optional Propertyها بیش از حد استفاده نکنید!!!

****

خیلی‌ها رو می‌بینم از آپشنال پراپرتی تایپ ها در تایپ اسکریپت (نه آپشنال پراپرتی خود جی اس) بیش از حد استفاده می‌کنن!
فقط و فقط شما حق استفاده از آپشنال پراپرتی تایپ هارو در موارد خاص دارید! در غیراین صورت یه جای کار کدتون می‌لنگه، کد کثیفه، آشغاله!!

ویژگی‌های اختیاری با `?` می‌توانند حالت‌های بیشتری برای یک آبجکت ایجاد کنند و باعث شوند مصرف‌کننده مجبور باشد نبودن ویژگی را نیز مدیریت کند.

```
interface Subscription {
  isSubscribed: boolean;
  expirationDate: Date;
}
```

در این تعریف، هر اشتراک باید وضعیت اشتراک و تاریخ انقضا داشته باشد.

اما اگر این ویژگی‌ها اختیاری باشند:

```
interface Subscription {
  isSubscribed?: boolean;
  expirationDate?: Date;
}
```

ممکن است هر دو ویژگی وجود نداشته باشند، حتی اگر چنین حالتی از نظر منطق برنامه معتبر نباشد.

بنابراین، فقط زمانی از `?` استفاده کنید که اختیاری بودن ویژگی بخشی واقعی از مدل داده باشد.

وجود بی مورد آپشنال تایپ ها باید در نظر داشته باشدی دولوپر بعدی یا حتی خودتون موقع استفاده از اون تایپ چون آپشنال هستش، بی خبر باشید که برای فلان‌جا اصلا باید پرارتی فلان رو ست کنید!!

مثلا برای (**به‌جای Union کردن Interfaceها، از Union Type استفاده کنید!***) که در چند بست پرکتیس قبلی گفته شد، یکی از کار هایی اشتباهی که اکثر تازه‌کار مرتکبش می‌شن اینه که درواقع باید 3تا یونیون تایپ داشته باشن کمه تو هرکدومشون یسری پراپرتی هایی باید وجود داشته باشه که تو دیگری نباید وجود داشته باشه، میان یه تایپ لی تعغریف می‌کنن و هر پراپرتی که تو ذهنشون مشخص کردن باید برای هر وضعیت باشه رو آپشنال می‌کنن! خوب دولوپر بعدی اصلا باید از کجا بفهمه بریا فلان وضعیت باید فلان تایپ آپشنال هم ست کنه؟ تایپ اسکریپت هم که هشداری عملا بهش نمیده! 🤦

از Discriminated Union برای مدل‌سازی حالت‌ها استفاده کنید!

وقتی چند نوع اعلان ساختارهای متفاوتی دارند، می‌توانید یک ویژگی مشترک مانند `type` تعریف کنید تا هر حالت به‌وضوح مشخص شود.

```ts
type NotificationType = "text" | "icon" | "action";

interface BaseNotification {
  type: NotificationType;
}

interface TextNotification extends BaseNotification {
  type: "text";
  message: string;
}

interface IconNotification extends BaseNotification {
  type: "icon";
  iconUrl: string;
}

interface ActionNotification extends BaseNotification {
  type: "action";
  message: string;
  actionText: string;
  onClick: () => void;
}

type TNotification =
  | TextNotification
  | IconNotification
  | ActionNotification;
```

در این ساختار، هر نوع اعلان ویژگی‌های متناسب با خودش را دارد و `type` به TypeScript کمک می‌کند حالت موردنظر را تشخیص دهد.

نوع بازگشتی توابع را صریح مشخص کنید!***

بهتر است در توابع مهم، به‌خصوص توابعی که بخشی از API داخلی پروژه یا منطق اصلی برنامه هستند، نوع بازگشتی را صریح مشخص کنید.

```
function getUserName(user: User): string {
  return user.name;
}
```

این کار قرارداد تابع را واضح‌تر می‌کند، خوانایی کد را افزایش می‌دهد و باعث می‌شود TypeScript نوع مقدار بازگشتی را نیز بررسی کند.

تفاوت `satisfies` با Type Annotation

```ts
type Point = [number, number];

const tehran = {
  location: [-73.75, 42.65],
} satisfies Record<string, Point>;
tehran.location[0] = 10;

const karaj: Record<string, Point> = {
  location: [-73.75, 42.65],
};
karaj.location2 = [47, 51];
```

وقتی از `satisfies` استفاده می‌کنیم، تایپ دقیق خود عبارت حفظ می‌شه؛ بنابراین در مثال اول، تایپ `tehran` به `Record<string, Point>` تبدیل نمی‌شه و نمی‌تونیم پراپرتی جدیدی مثل `location2` بهش اضافه کنیم.

اما در مثال دوم، تایپ متغیر مستقیما `Record<string, Point>` تعریف شده؛ بنابراین می‌تونیم پراپرتی‌های جدیدی با کلیدهای رشته‌ای و مقدارهای سازگار با `Point` بهش اضافه کنیم.

نکته: `satisfies` سازگاری تایپ رو بررسی می‌کنه، اما تایپ عبارت رو با تایپ سمت راست جایگزین نمی‌کنه.

🤔 یه رفتار عجیب در TypeScript! (چالشی برای کامپایلر تایپ اسکریپت)

کد های زیر رو با دقت نگاه کنید.

```
class Car {
  drive() {}
}

class Truck extends Car {
  load() {}
}
```

حالت اول:

```
const useCar = (car: Car | Truck) => {
  if (car instanceof Car) {
    // car.drive(100);
  } else if (car instanceof Truck) {
    // car.load(200);
  }
};
```

حالت دوم:

```
const useCar = (car: Car | Truck) => {
  if (car instanceof Truck) {
    // car.drive(100);
  } else if (car instanceof Car) { // !ارور اینجا رخ میده
   //  car.load(200);
  }
};
```

در هر دو حالت، پارامتر `car` از نوع `Car | Truck` هست؛ اما در یکی از حالت‌ها ممکنه TypeScript اون رو `never` تشخیص بده و خطای `ts(2358)` نمایش بده.

اروری که در حالت اول می‌گیریم:

```
The left-hand side of an 'instanceof' expression must be of type 'any', an
object type or a type parameter. ts(2358)
(parameter) car: never
```

سوال: چه چیزی می‌تونه باعث این رفتار بشه؟ آیا ترتیب شرط‌های `instanceof` به‌تنهایی دلیلشه؟ 🤔

.

.

.

.

.

.

.

.

.

.

.

.

🤔 چرا ترتیب شرط‌های `instanceof` مهمه؟

*چون `Truck` از `Car` ارث‌بری می‌کنه، هر نمونه از `Truck` شرط `instanceof Car` رو هم پاس می‌کنه.

استخراج تایپ اعضای آرایه در TypeScript

در TypeScript می‌تونیم با استفاده از **Indexed Access Type**، نوع یک عضو مشخص از آرایه یا نوع تمام اعضای اون رو استخراج کنیم.

```ts
const statuses = ["published", "draft", "archived"] as const;

type FirstStatus = (typeof statuses)[0];
// "published"

type SecondStatus = (typeof statuses)[1];
// "draft"

type AllStatuses = (typeof statuses)[number];
// "published" | "draft" | "archived"
```

- `[0]` نوع عضو اول رو استخراج می‌کنه.
- `[1]` نوع عضو دوم رو استخراج می‌کنه.
- `[2]` نوع عضو سوم رو استخراج می‌کنه.
- `[number]` نوع تمام اعضای آرایه رو به‌صورت یک **Union Type** درمیاره.

چرا از `number` استفاده می‌کنیم؟

در اینجا `number` به معنی اجرای حلقه یا پیمایش آرایه نیست. چون اندیس‌های آرایه از نوع `number` هستن، استفاده از `[number]` باعث می‌شه TypeScript نوع اعضای آرایه رو به‌صورت یک Union استخراج کنه. (**البته برای درک راحت‌تر می‌تونید کلا یسری چیزها رو در تایپ اسریپت به صورت حلقه تفسیر کنید! درحالی که هیچ شباهتی به سینتکس حلقه‌ها ندارند!**)

به عبارت ساده‌تر، `[0]` محدوده مشخصی داره و فقط نوع عضو اول رو می‌گیره؛ اما `[number]` نوع تمام اعضای آرایه رو استخراج می‌کنه.

نکته: استفاده از `as const` باعث می‌شه مقادیر آرایه به‌صورت Literal Type حفظ بشن؛ در نتیجه به‌جای `string`، نوع‌های دقیق‌تری مثل `"published"` و `"draft"` رو خواهیم داشت.

روش پیشنهادی من برای طراحی تایپ ها در تایپ اسکریپت

من همیشه از حداقل‌ها شروع می‌کنم. همون اول کار، خودتون رو غرق تعریف تایپ‌ها و ساختارهای پیچیده نکنید!

از هممون ابتدا تمام پراپرتی‌های احتمالی حالت های مثلا کامپوننت Button مختلف و تایپ‌های پیچیده رو تعریف نکنید. ممکنه در نهایت کلی تایپ داشته باشید که حتی بهشون نیازی پیدا نکنید! (حتی اگه دیزاین سیستم فیگما دارید اول ببینید چه حالت‌هایی رو فقط در دیزاین استفاده کرده‌اند! نه اینکه بیایی تمام حالت‌های احتمالی آینده رو هم اضافه بکنی! - YAGNI*)

ما از TypeScript استفاده می‌کنیم تا در زمان توسعه، با کمک سیستم تایپ بتونیم خطاها رو زودتر تشخیص بدیم و کد مطمئن‌تری بنویسیم.

اما وقتی از همون اول بیش از حد درگیر تایپ‌های پیچیده می‌شید، ممکنه به‌جای حل مسئله اصلی، تمام تمرکزتون روی محدود کردن تایپ‌ها و مدیریت ساختارهای پیچیده قرار بگیره. حتی ممکنه تعداد پراپرتی‌ها و حالت‌های مختلف اون‌قدر زیاد بشه که برای اضافه کردن هر قابلیت جدید، مجبور بشید چندین تایپ رو تغییر بدید!

*این نکته رو قاب بگیرید و جلوی ذهنتون بذارید:

لطفا لطفا از همون اول، برای همه‌چیز تایپ تعریف نکنید!

حتی اگه Design System دارید یا طرح‌های Figma در اختیارتونه، لازم نیست تمام حالت‌های احتمالی رو از همون ابتدا وارد کد کنید. اگه در طراحی فقط از دو یا سه حالت استفاده شده، فعلا همون‌ها رو پیاده‌سازی کنید. هر وقت نیاز واقعی به حالت جدیدی داشتید، به ساختار اضافه‌اش کنید. (YAGNI*)

پیشنهاد من اینه که اول مسئله رو حل کنید و بعد، به‌مرور و متناسب با نیاز پروژه، تایپ‌های TypeScript رو به کد اضافه کنید. لازم نیست قبل از نوشتن کد، تمام جزئیات تایپی رو مشخص کرده باشید!

برای پارامترهای تابع هم اگه یک پارامتر اختیاریه و مقدار پیش‌فرض مشخصی داره، می‌تونید همون ابتدا مقدار پیش‌فرض رو براش تعیین کنید. این کار کمک می‌کنه در بسیاری از موارد نیازی به بررسی جداگانه `undefined` داخل بدنه تابع نداشته باشید.

حالا می‌خوام یک متد خوب رو بهتون معرفی کنم: **متد Step by Step!** روشی که خودم همیشه سعی می‌کنم باهاش جلو برم.

مرحله اول: اول کد رو بنویسیم!

فرض کنید می‌خوایم تم یک اپلیکیشن رو تغییر بدیم و تابعی به نام `applyTheme` داشته باشیم که تنظیمات دلخواه رو دریافت می‌کنه.

```
const defaultThemeConfig = {
  darkMode: false,
  fontSize: 16,
};

const applyTheme = (configs) => {
  const allConfigs = {
```

```
    ...defaultThemeConfig,
    ...configs,
  };

  // ...
};
```

در این مرحله، منطق اصلی رو نوشتیم؛ اما هنوز تایپ پارامتر `configs` مشخص نیست و TypeScript در صورت فعال بودن `noImplicitAny` برای اون خطا می‌ده.

مرحله دوم: تایپ تنظیمات رو تعریف کنیم

خب، بیایم تایپ تنظیمات رو مشخص کنیم:

```
type ThemeConfig = {
  darkMode: boolean;
  fontSize: number;
};

const defaultThemeConfig: ThemeConfig = {
  darkMode: false,
  fontSize: 16,
};



const applyTheme = (configs?: ThemeConfig) => {
```

```
  const allConfigs = {
    ...defaultThemeConfig,
    ...configs,
  };
};

applyTheme({
  darkMode: true,
  fontSize: 18,
});
```

حالا تایپ پارامتر مشخص شده و خطای مربوط به `any` هم برطرف شده!

اما به نظرتون مشکل این کد چیه؟ کمی نفکر کنید...

.

.

.

.

.

.

.

.

.

مشکل اینجاست که خود پارامتر `configs` اختیاریه، اما پراپرتی‌های داخل `ThemeConfig` همچنان اجباری هستن!

یعنی این کد خطا می‌ده:

```
applyTheme({
  darkMode: true,
}); // Error
```

چون `fontSize` رو ارسال نکردیم؛ در حالی که هدفمون اینه که کاربر بتونه فقط بخشی از تنظیمات رو تغییر بده و بقیه از مقادیر پیش‌فرض استفاده کنن.

مرحله سوم: از Partial استفاده کنیم

راه اول اینه که یک تایپ جدید برای تنظیمات اختیاری بسازیم:

```
type ThemeConfig = {
  darkMode: boolean;
  fontSize: number;
};

type PartialThemeConfig = {
  darkMode?: boolean;
  fontSize?: number;
};

const defaultThemeConfig: ThemeConfig = {
  darkMode: false,
  fontSize: 16,
};

const applyTheme = (configs?: PartialThemeConfig) => {
  const allConfigs = {
    ...defaultThemeConfig,
    ...configs,
  };

  console.log(allConfigs);
};
```

```
applyTheme({
  fontSize: 20,
});
```

حالا می‌تونیم فقط `fontSize` رو تغییر بدیم و مقدار `darkMode` همون `false` باقی می‌مونه.

اما یک مشکل داریم!

اگه بخوایم پراپرتی جدیدی به تنظیمات اضافه کنیم، باید تایپ‌ها رو در چند جای مختلف تغییر بدیم. این کار در پروژه‌های بزرگ‌تر می‌تونه باعث تکرار و نگهداری سخت‌تر کد بشه.

مرحله چهارم: از Utility Type آماده استفاده کنیم

```
type ThemeConfig = {
  darkMode: boolean;
  fontSize: number;
};

const defaultThemeConfig: ThemeConfig = {
  darkMode: false,
  fontSize: 16,
};

const applyTheme = (configs?: Partial<ThemeConfig>) => {
  const allConfigs = {
    ...defaultThemeConfig,
    ...configs,
  };

  console.log(allConfigs);
};
```

```
applyTheme({
  fontSize: 20,
});
```

با استفاده از `Partial<ThemeConfig>`، تمام پراپرتی‌های `ThemeConfig` اختیاری می‌شن و دیگه نیازی به تعریف تایپ جداگانه نداریم.

اگه بعدا پراپرتی جدیدی به `ThemeConfig` اضافه کنیم، `Partial` به‌صورت خودکار اون رو هم اختیاری می‌کنه.

مرحله پنجم: یک قدم جلوتر؛ از typeof استفاده کنیم!

اما هنوز می‌تونیم تعریف تایپ رو ساده‌تر کنیم.

بیایم به‌جای تعریف جداگانه `ThemeConfig`، تایپ تنظیمات رو از روی خود آبجکت پیش‌فرض استخراج کنیم:

```
const defaultThemeConfig = {
  darkMode: false,
  fontSize: 16,
  primaryColor: "#333",
  // ... other properties
};

const applyTheme = (
  configs?: Partial<typeof defaultThemeConfig>,
) => {
  const allConfigs = {
    ...defaultThemeConfig,
    ...configs,
};

console.log(allConfigs);
};
```

```ts
applyTheme({
  fontSize: 20,
});
```

اینجا دیگه نیازی به تعریف `ThemeConfig` نداریم!

عبارت `typeof defaultThemeConfig` تایپ استنباط‌شده آبجکت رو استخراج می‌کنه و `Partial` تمام پراپرتی‌های اون تایپ رو اختیاری می‌کنه.

حالا اگه پراپرتی جدیدی به `defaultThemeConfig` اضافه کنیم، تایپ پارامتر تابع هم به‌صورت خودکار به‌روز می‌شه.

و تمام! 😁

چندین نکته مهم

- لازم نیست همیشه از همون ابتدا سراغ این روش برید؛ اول نیاز واقعی رو بررسی کنید.
- اگه تایپ تنظیمات قراره در بخش‌های مختلف پروژه استفاده بشه، تعریف یک `type` یا `interface` مستقل ممکنه انتخاب مناسب‌تری باشه.
- اگه تایپ فقط از یک آبجکت مشخص استخراج می‌شه و نیاز به تعریف مستقل نداره، `typeof` می‌تونه از تکرار جلوگیری کنه.
- `Partial` فقط پراپرتی‌های سطح اول رو اختیاری می‌کنه و به‌صورت پیش‌فرض بازگشتی نیست.
- `typeof` تایپ رو در زمان توسعه استخراج می‌کنه؛ این قابلیت به‌معنای اعتبارسنجی داده‌ها در زمان اجرا نیست.

- این روش همیشه بهترین روش ممکن نیست؛ انتخاب درست، به ساختار و نیاز واقعی پروژه بستگی داره.

اول مسئله رو حل کنید، بعد به‌اندازه نیاز براش تایپ طراحی کنید.

نه کمتر از چیزی که برای ایمنی کد لازمه و نه بیشتر از چیزی که نگهداری پروژه رو سخت کنه.

بلاخره تایپ‌ها رو محدود و دقیق تنظیم بکنیم یا نه؟!

تا اینجا ما یسری جاها گفتنیم تاجایی که میشه محدود تایپ بنویسید که اشک کده در بیاد ;) اما یکم قبل‌تر گفتیم خیلی هم محدود و دقیق کار نکنید!!

اما این نوشتن تایپ ها هم نباید خیلی ریز و با دقت بالا باشند! یسری جاها واقعا نیازه که بذارید generalتر باشند!

واقعیتش اینه فرمول و قانون خاصی براش وجود نداره؛ یسری چیزا رو باید طبق تجربه منوحهش بشید که فلان چیز داخل فلان موقع خوبه یا بده 🤷 ولی به طور کلی بذارید تایپ اینفرنس کار خودشو بکنه و فقط در موقعیت هایی که نوشتن تایپ به خوانایی یا مواقعی که اینفرنس کاری از دستش بر نمیاد مثل کار با داده های بک اند، اونجا خودتون تایپ ها رو صریحا مشخص کنید!*

اجازه بدید تا حد امکان تایپ اسکریپت تایپ ها را استخراج کند. فقط در شرایطی که واقعا لازم است تایپ ها را صریحا تعریف کنید!

2 3 کانسپت مهم رو درنظرگرفتم که آخر این کتابچه بهشون بپردازیم که ملکه ذهنتون بشه!

درواقع مهم‌ترین مباحث این کتابچه این 3 بخش آخر بودند! که تو این بخش مهم‌ترینشون رو توضیح میدم...

همانطور که قبلا اشاره ریزی برای این موضوع شده بود، به‌طور کلی دولوپر ها رو به 2دسته تبدیل می‌کنه!

یسری‌ها میگن برای خوانایی همیشه‌ی خدا تایپ‌ها رو صریحا مشخص کنید، اما یسری ها از جمله خودم بر این باوریم که بهتره بذاریم خود تایپ اسکریپت تایپ ها رو استنتاج کنه!!

وقتی که یسری جاها وقتی تایپ اسکریپت می‌تونه اینفرنس کار کنه، چرا ما بیایم باز هم انوتیشن ست کنیم؟! فکر نمی‌کنید کد یکم الکی شلوغ میشه؟ اضافه کاری نیست؟ بهتره فقط جاهایی که نیازه و به خوانایی کمک می‌کنه صراحتا ست کنید!!*

بهتره تا جای ممکن اجازه بدیم TypeScript تایپ‌ها رو از روی مقدارها و Context استنباط کنه. فقط زمانی تایپ‌ها رو صریح مشخص کنیم که واقعا بهشون نیاز داریم یا باعث خوانایی و دقت بیشتر کد می‌شن.

1. برای متغیرهای محلی، به Type Inference اعتماد کنید!

```
let score: number = 10;
```

نیازی به نوشتن تایپ در کد بالا نیست!

```
let score = 10;
```

این روش ساده‌تره و از تکرار اطلاعات و مکررات جلوگیری می‌کنه.

2. تایپ آبجکت رو بی‌دلیل دوباره تعریف نکنید!

```
const course: {
  title: string;
  subTitle: string;
  students: number;
} = {
  title: "WebSocket",
  subTitle: "Learn WebSocket",
  students: 100,
};
```

در این مثال، هم ساختار تایپ رو مشخص کردیم و هم آبجکت رو با همون ساختار مقداردهی کردیم. TypeScript می‌تونه تایپ رو از روی خود آبجکت استنباط کنه؛ بنابراین این Type Annotation معمولا اضافه است.

```
const course = {
  title: "WebSocket",
  subTitle: "Learn WebSocket",
  students: 100,
};
```

البته اگر این ساختار قراره در چند جای برنامه استفاده بشه، تعریف `interface` یا `type` می‌تونه انتخاب مناسبی باشه.

نکته: تکرار غیرضروری اطلاعات در کد رو اصطلاحا *Noise* (نویز/سر و صدا) می‌گن؛ یعنی کدی که بدون اضافه کردن ارزش واقعی، خوانایی رو کاهش می‌ده.

3. برای موجودیت‌ها (Entityها) و قراردادهای مشخص، Type Annotation مفید است!

وقتی یک آبجکت نماینده یک موجودیت مشخص در برنامه است و باید ساختار معینی داشته باشه، بهتره تایپش رو صریح تعریف کنیم.

```
interface Course {
  id: string;
  title: string;
}

const course: Course = {
  id: "50",
  title: "WebSocket",
};
```

اینجا مشخص کردن `Course` کمک می‌کنه ساختار مورد انتظار حفظ بشه و اشتباهاتی مثل اختصاص دادن `number` به `id` شناسایی بشن.

این کار به‌خصوص برای داده‌های دامنه برنامه، ورودی‌ها و خروجی‌های API و قراردادهای مشترک مفیده.

4. برای پارامترهای تابع، تایپ‌ها رو مشخص کنید!

```
type Point = [number, number];
const addPoints = (a: Point, b: Point): Point ⇒ {
```

4. برای پارامترهای تابع، تایپ‌ها رو مشخص کنید!

```
  return [a[0] + b[0], a[1] + b[1]];
};
```

تایپ پارامترهای تابع رو معمولا باید صریح مشخص کنیم؛ چون TypeScript نمی‌تونه در همه شرایط نوع ورودی مورد انتظار رو از روی نحوه استفاده از تابع تشخیص بده.

5. برای Return Type تابع هم Type Annotation در نظر بگیرید!

TypeScript در بسیاری از موارد می‌تونه نوع خروجی تابع رو استنباط کنه؛ اما مشخص کردن صریح Return Type، به‌خصوص برای توابع عمومی، توابع پیچیده و بخش‌هایی که قرارداد مشخصی دارن، می‌تونه مفید باشه.

```
const getCourseTitle = (course: Course): string ⇒ {
  return course.title;
};
```

در این مثال، `string` نوع خروجی مورد انتظار رو مشخص می‌کنه و اگر بعدا خروجی تابع با این قرارداد سازگار نباشه، TypeScript خطا می‌ده.

چرا TypeScript یک مقدار `string` را به Literal Type قبول نمی‌کند؟

فرض کنید یک تابع فقط مقدار `"ali"` را می‌پذیرد:

```
function greet(name: "ali") {
  console.log(`Hello ${name}`);
}
```

حالا یک متغیر داریم که مقدارش `"ali"` است، اما تایپ آن `string` است:

```
const name: string = "ali";

greet(name); // Error
```

چرا خطا می‌گیریم؟

چون TypeScript بین مقدار فعلی یک متغیر و تایپ آن تفاوت قائل می‌شود.

ما می‌دانیم که مقدار `name` برابر `"ali"` است، اما تایپ آن `string` است؛ یعنی از دید سیستم تایپ، این متغیر می‌تواند هر رشته‌ای را نگه دارد.

در مقابل، تابع فقط `"ali"` را می‌پذیرد. بنابراین، TypeScript نمی‌تواند تضمین کند که مقدار ورودی حتما همان `"ali"` است.

اما اگر متغیر را بدون Type Annotation تعریف کنیم:

```
const name = "ali";

greet(name); // OK
```

این بار TypeScript تایپ متغیر را `"ali"` استنباط می‌کند؛ چون متغیر `const` است و امکان انتساب مجدد به آن وجود ندارد.

نکته اصلی: برابر بودن مقدار دو متغیر به معنای یکسان بودن تایپ آن‌ها نیست. TypeScript برای بررسی سازگاری، به تایپ استنباط‌شده یا اعلام‌شده توجه می‌کند، نه فقط مقدار فعلی متغیر.

تا جای ممکن از تایپ‌های محدودتر استفاده کنید!

تایپ `string` در بسیاری از سناریوها بیش از حد کلی است. بهتره قبل از تعریف تایپ‌ها، مشخص کنیم هر مقدار واقعا چه حالت‌هایی می‌تونه داشته باشه و تا جای ممکن از تایپ‌های محدودتر استفاده کنیم.

نکات مهم:

- به‌جای استفاده مستقیم از `string`، وقتی مقدارها مشخص و محدود هستن، از **String Literal Union** استفاده کنید؛ مثلا `"success" | "error" | "loading"`.
- برای محدود کردن مقدارهای مجاز، به‌جای پذیرش هر رشته‌ای، تایپی تعریف کنید که فقط حالت‌های معتبر رو بپذیره.
- در پارامترهای تابع هم می‌تونیم با استفاده از **Generics** و `keyof` تایپ‌ها رو دقیق‌تر کنیم.

```
function extract<T, K extends keyof T>(list: T[], key: K): T[K][] {
  return list.map((item) => item[key]);
}
```

در این مثال، `K extends keyof T` باعث می‌شه فقط کلیدهای معتبر `T` قابل استفاده باشن و `T[K][]` هم نوع مقادیر مربوط به همون کلید رو حفظ کنه.

هدف اینه که به‌جای پذیرفتن هر مقداری، تا جای ممکن محدوده مقادیر مجاز رو با تایپ‌ها مشخص کنیم.

استفاده از Template Literal Types

با ترکیب `Template Literal Types` و `Union Types` می‌توان تایپ‌هایی ساخت که فقط الگوهای مشخصی از رشته‌ها را قبول کنند.

```
type Vertical = "top" | "center" | "bottom";
type Horizontal = "left" | "center" | "right";

type Positions = Exclude<
  `${Vertical}-${Horizontal}`,
  "center-center"
>;

const position: Positions = "top-right"; // معتبر
const invalidPosition: Positions = "center-center"; // خطا
```

در این مثال، فقط موقعیت‌های ترکیبی معتبر پذیرفته می‌شوند و `center-center` با `Exclude` حذف شده است.

مثال جذابی دیگر:

```
type MessagesType = "error" | "notification" | "res";
type Languages = "fa" | "en" | "es" | "tr";

type Messages = ${MessagesType}_${Languages};

export type AllTypes = {
toTranslate: Messages;
};
```

محدود کردن پراپرتی‌ها با `never`

با استفاده از `never` که در مجموعه‌ها به معنی توهی هستش، در یک Union Type می‌توان مشخص کرد که یک پراپرتی در یک حالت نباید وجود داشته باشد.

```
type PngIconType = {
  png: boolean;
  svg?: never;
};

type SvgIconType = {
  png?: never;
  svg: boolean;
};

type IconProps = PngIconType | SvgIconType;

const pngIcon: IconProps = { png: true }; // معتبر
const svgIcon: IconProps = { svg: true }; // معتبر

const invalidIcon: IconProps = {
  png: true,
  svg: true,
}; // خطا
```

در این مثال، `never` مانع استفاده هم‌زمان از `png` و `svg` می‌شود.

چرا `readonly unknown[]` آرایه‌های معمولی را هم می‌پذیرد؟🧐📏

در TypeScript، سازگاری تایپ‌ها بر اساس رابطه زیرمجموعه بودن مجموعه مقادیر قابل‌قبول تعریف نمی‌شود، بلکه بر اساس سازگاری ساختاری تایپ‌ها بررسی می‌شود. تایپ `readonly unknown[]` یک قرارداد برای آرایه‌هایی است که می‌توان عناصر آن‌ها را خواند، اما از طریق این مرجع نمی‌توان عناصر را تغییر داد.

از آنجا که آرایه معمولی مانند `number[]` نیز این قرارداد را رعایت می‌کند، با `unknown[]` سازگار است. بنابراین، در عبارت `T extends readonly unknown[]`، هم آرایه‌های معمولی و هم آرایه‌های فقط‌خواندنی پذیرفته می‌شوند.

به بیان ساده، `readonly` در اینجا محدودیت تغییرپذیری را به تایپ ورودی تحمیل نمی‌کند؛ بلکه مشخص می‌کند که برای سازگاری، کافی است تایپ ورودی قابلیت‌های خواندن یک آرایه را فراهم کند. در نتیجه، این محدودیت از پذیرش آرایه‌های معمولی جلوگیری نمی‌کند و تایپ اصلی `T` نیز تغییر نمی‌کند.

```ts
type ArrayType<T extends readonly unknown[]> = T;

type Normal = ArrayType<number[]>; // number[]
type Readonly = ArrayType<readonly number[]>; // readonly number[]
```

در این مثال، هر دو تایپ پذیرفته می‌شوند و تایپ اصلی `T` نیز حفظ می‌شود. `readonly` در اینجا آرایه معمولی را فقط‌خواندنی نمی‌کند؛ بلکه اجازه می‌دهد آرایه‌های فقط‌خواندنی نیز با این محدودیت سازگار باشند.

اما اگه `readonly` رو حذف کنیم، آرایه‌های معمولی پذیرفته می‌شن، اما آرایه‌های `readonly` مثل Tupleهای ساخته‌شده با `as const` دیگه پذیرفته نمی‌شن.

```ts
type ArrayType<T extends unknown[]> = T;

type A = ArrayType<number[]>; // ✅
type B = ArrayType<readonly [1, 2, 3]>; // ❌ خطا
```

با اضافه کردن `readonly`:

```ts
type ArrayType<T extends readonly unknown[]> = T;

type A = ArrayType<number[]>; // ✅
type B = ArrayType<readonly [1, 2, 3]>; // ✅
```

نتیجه: `readonly` باعث می‌شه محدودیت ما هم آرایه‌های معمولی و هم آرایه‌های فقط‌خواندنی رو بپذیره.

دنیای زهر ماری تایپ‌اسکریپت هم جالبه ها😇

تنظیمات و کانفیگ‌های کاربردی کامپایلر

برای بخش پایانی این کتابچه در نظر گرفته‌ام که تنظیمات و کانفیگ های کامپایلر تایپ اسکریپتی که خودم همیشه در پروژه هام استفاده می‌کنم را در اختیارتون قرار بدم.

همواره پیشنهاد می‌شود که پروژه هاتون رو با vite یا با سایر ابزار های مشابه اجرا کنید!

برای دریافت ۳ فایل کانفیگ و تنظیمات، به این پست در کانال تلگرام ما مراجعه کنید:

https://t.me/SabziDevCode/87

منابع

- تجربیات شخصی خود نویسنده (ابوالفضل سبزمحمدی)
- عباس سپهوند (منبع اصلی)

و درنهایت تشکر از

تمام کسانی که در شکل‌گیری این کتابچه و pdf کمک کرده‌اند.

پایان این کتابچه، آغاز یادگیری‌های تازه است.

ممنون از وقتی که برای یادگیری خودت گذاشتی ❤️

Keep learning • Keep growing

</ Abolfazl>